

Modern LLM and AI Algorithms — Interview Guide

This guide explains modern tokenization, Transformer attention, language-model objectives, alignment, and parameter-efficient fine-tuning from fundamentals to interview depth. Every chapter follows the same 20-section structure and includes formulas, practical code, failure modes, evaluation guidance, interview questions, exercises, and project ideas.

Tokenization

1. Overview

Tokenization converts raw text into smaller units called tokens. Tokens may be characters, words, subwords, bytes, or special symbols. It is useful because neural networks cannot directly process strings; they need integer IDs that can be mapped to embeddings. Tokenization is used in every NLP and LLM pipeline: search, chatbots, translation, summarization, RAG, sentiment analysis, and code generation.

2. Intuition

Think of tokenization as cutting a sentence into model-readable pieces. For example, "unhappiness" may become ["un", "happiness"] or ["un", "happy", "ness"]. Subword tokenization helps models understand rare words by composing them from known pieces.

3. Prerequisites

- Python strings and Unicode basics
- Vocabulary, integer encoding, and one-hot vectors
- Basic NLP preprocessing
- Probability and frequency counts
- Transformer input pipelines

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Vocabulary	Set of known tokens	Defines representable text	["the", "cat", "##s"]	Vocabulary size vs memory
Token ID	Integer assigned to token	Model input is integer IDs	"cat" -> 3457	IDs are not semantic by themselves
Special tokens	Control tokens	Mark padding, BOS, EOS, masks	[PAD], <eos>	Padding masks and generation stop
Subword tokens	Pieces of words	Handles rare/OOV words	"playing" -> "play", "ing"	BPE/WordPiece/Unigram

Concept	Meaning	Why it matters	Example	Interview angle
Byte tokens	Byte-level pieces	Robust to any text	GPT-style byte BPE	Unicode handling

5. Algorithm / Working Process

1. Normalize text if the tokenizer requires it.
2. Split text into candidate units such as bytes, characters, or words.
3. Apply a learned segmentation algorithm such as BPE, WordPiece, or Unigram.
4. Map each token to an integer ID from the vocabulary.
5. Add special tokens if required.
6. Pad or truncate to the model's maximum length.
7. Return `input_ids` and usually `attention_mask`.

Input: raw text.

Output: token IDs and masks.

Training process: learn vocabulary/merge rules from a corpus.

Inference process: apply fixed tokenizer rules learned during training.

6. Mathematical Foundation

Tokenization is mostly algorithmic, but training uses frequency and likelihood.

BPE starts from characters/bytes and repeatedly merges the most frequent pair:

```
merge* = argmax_(a,b) count(a,b)
```

Unigram tokenization chooses a segmentation `S` that maximizes likelihood:

```
P(text) = product over token t in S of P(t)
best_S = argmax_S P(text)
```

Important trade-off:

```
larger vocabulary -> shorter sequences but larger embedding table
smaller vocabulary -> longer sequences but better rare-word coverage
```

7. Practical Implementation

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
texts = [
    "Transformers changed modern NLP.",
    "Tokenization handles unseen words using subwords."
]

batch = tokenizer(
    texts,
    padding=True,
    truncation=True,
    max_length=16,
    return_tensors="pt"
)

print(batch["input_ids"])
print(batch["attention_mask"])
print(tokenizer.convert_ids_to_tokens(batch["input_ids"][0]))
```

8. Code Explanation

`AutoTokenizer` loads the exact tokenizer used by the pretrained model. `padding=True` makes all examples in the batch the same length. `truncation=True` prevents inputs from exceeding the model limit. `attention_mask` marks real tokens with `1` and padding with `0`.

9. Training / Evaluation

For tokenizer training, use a representative corpus. Evaluate average tokens per word, unknown-token rate, compression ratio, domain coverage, and downstream task performance. Poor tokenization can hurt domain tasks such as medical NLP, Indian-language NLP, legal search, and code models.

10. Complexity and Cost

Tokenization is usually CPU-bound. Runtime is roughly linear in text length, though algorithms differ. Memory cost is mainly vocabulary size and embedding table size:

```
embedding_parameters = vocabulary_size * embedding_dim
```

11. Common Use Cases

- Preparing text for BERT, GPT, T5, LLaMA, and other LLMs
- RAG chunk counting
- Chatbot prompt budgeting
- Search indexing
- Code model preprocessing

12. Common Mistakes

- Using a tokenizer different from the model's tokenizer
- Counting words instead of tokens for context limits

- Forgetting padding masks
- Truncating away labels or key context
- Training a tokenizer on a corpus that does not match the target domain

13. Edge Cases / Limitations

Tokenizers may split names, URLs, emojis, code, math, and low-resource-language text poorly. Tokenization can also introduce bias because some languages require more tokens for the same meaning, increasing cost and reducing effective context.

14. Variations

Variation	What changes	When to use	Placement value
Word tokenization	Splits by words	Classical NLP	Basic
Character tokenization	Uses characters	No OOV issue	Good for spelling/noisy text
BPE	Frequent pair merges	GPT-style models	Very important
WordPiece	Likelihood-inspired subwords	BERT-style models	Very important
SentencePiece/Unigram	Language-independent segmentation	Multilingual models	Important
Byte-level BPE	Works on bytes	Robust LLM tokenization	Important

15. Related Topics

Tokenization connects to embeddings because token IDs index embedding vectors. It connects to context window because token count, not word count, determines length. It connects to prompting because concise prompts save tokens. It connects to RAG because chunk sizes are usually token-based.

16. Interview Questions

1. What is tokenization?
It converts text into model-readable tokens and token IDs.
2. Why do LLMs use subword tokenization?
It handles rare words while keeping vocabulary manageable.
3. What is an OOV token?
An out-of-vocabulary token that the tokenizer cannot represent directly.
4. Why must the tokenizer match the model?
Token IDs map to learned embeddings; mismatched IDs mean wrong inputs.
5. What does `attention_mask` do?
It prevents the model from attending to padding tokens.
6. BPE vs WordPiece?
BPE merges frequent pairs; WordPiece chooses merges with a likelihood-style criterion.
7. Why are emojis and code difficult?
They may split into many tokens and waste context.

8. What happens if text is truncated?

The model loses information beyond the maximum length.

9. How does vocabulary size affect cost?

Larger vocabularies increase embedding and output projection parameters.

10. Why does tokenization matter in RAG?

Chunk size and prompt length must fit within the context window.

17. Practice Tasks

- Tokenize the same paragraph with BERT and GPT-2 tokenizers and compare token counts.
- Train a tiny BPE tokenizer on a custom corpus.
- Debug a classification model where labels are truncated accidentally.
- Measure token cost for different prompt templates.
- Compare tokenization quality for English, Hindi, code, and URLs.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Token Cost Analyzer	Estimates LLM prompt cost	Python, HF tokenizers	User prompts	Useful AI engineering tool
Domain Tokenizer	Trains tokenizer for medical/legal text	SentencePiece	PubMed/legal docs	Shows NLP preprocessing depth
Multilingual Token Audit	Compares token efficiency by language	Python, pandas	FLORES/Wikipedia	Good fairness/LLM analysis

19. Quick Revision

- Key idea: text must become token IDs.
- Main formula: embedding parameters = $V * d$.
- When to use: every NLP/LLM system.
- Important metrics: token count, OOV rate, compression ratio.
- Common traps: wrong tokenizer, truncation, ignored padding mask.
- Interview one-liner: tokenization is the bridge between raw language and neural input IDs.

20. Final Cheat Sheet

Item	Summary
Definition	Splitting text into model-readable units
Input/output	Text -> token IDs + masks
Main steps	normalize, split, segment, map, pad/truncate
Hyperparameters	vocab size, max length, special tokens

Item	Summary
Metrics	token count, OOV rate, downstream score
Pros	handles text efficiently
Cons	language/domain bias, token inflation
Best use cases	all NLP and LLM pipelines

BPE

1. Overview

Byte Pair Encoding (BPE) is a deterministic subword tokenization algorithm. It begins with small symbols—usually characters or bytes—and repeatedly merges the most frequent adjacent pair. The learned merge list converts common strings into single tokens while leaving rare strings decomposable. GPT-2, RoBERTa, and many code models use byte-level BPE variants.

BPE matters because a word-only vocabulary cannot represent unseen words without an unknown token, while a character-only vocabulary creates long sequences. BPE offers a practical middle ground: frequent words may be one token, and rare words become reusable pieces.

2. Intuition

Suppose a corpus repeatedly contains low, lower, and lowest. Starting with characters, the pair l o may be frequent enough to merge into lo; later lo w becomes low. The model now processes low efficiently while still representing a new word such as lowland as low + l + a + n + d.

The merge table is a learned compression dictionary. It does not understand morphology, although frequent merges often resemble stems and suffixes.

3. Prerequisites

- Strings, tuples, dictionaries, and frequency counters
- Unicode characters versus UTF-8 bytes
- Vocabulary size and embedding lookup tables
- Greedy algorithms and corpus frequency statistics
- Basic tokenizer/model compatibility

4. Core Concepts

Concept	What it means	Why it matters	Simple example	Common interview angle
Base alphabet	Initial symbols	Guarantees coverage of representable inputs	l,o,w or byte values	Character BPE vs byte BPE

Concept	What it means	Why it matters	Simple example	Common interview angle
Pair frequency	Weighted count of adjacent symbols	Selects the next merge	$(1,o): 12$	Counts must use corpus frequencies
Merge rule	Replacement of a pair by one symbol	Builds longer tokens	$1 + o \rightarrow 1o$	Merge order is part of tokenizer state
Merge rank	Priority assigned by training order	Resolves overlapping merges at inference	$\text{rank}(1,o) < \text{rank}(1o,w)$	Why tokenization is deterministic
End-of-word marker	Optional boundary symbol	Prevents accidental cross-word merges	$low \text{ } \langle /w \rangle$	Classic BPE vs byte-level BPE
Byte-level mapping	Applies BPE after reversible byte encoding	Avoids [UNK] for arbitrary Unicode	emoji becomes bytes	Robustness versus token inflation

5. Algorithm / Working Process

Training:

1. Count corpus word frequencies.
2. Represent each word as characters or bytes, optionally adding an end marker.
3. Count all adjacent symbol pairs, weighted by word frequency.
4. Select the most frequent pair.
5. Merge every occurrence of that pair.
6. Record the merge and repeat until the vocabulary/merge budget is reached.

Encoding:

1. Convert new text to the same base symbols used in training.
2. Find applicable learned pairs.
3. Apply them in learned rank order until no merge applies.
4. Map resulting symbols to token IDs.

Decoding reverses token-to-string/byte mapping and concatenation. The model receives IDs; BPE itself does not create semantic vectors.

6. Mathematical Foundation

At iteration k , choose the pair with the largest weighted frequency:

$$(a^*, b^*) = \arg \max_{(a,b)} \sum_{w \in C} f(w) \text{count}_w(a, b)$$

where $f(w)$ is the corpus frequency of word w . Then replace each adjacent occurrence:

$$(\dots, a, b, \dots) \rightarrow (\dots, ab, \dots)$$

If the base vocabulary contains V_0 symbols and M merges are retained, the theoretical vocabulary has at most $V_0 + M$ entries, excluding special tokens. For an embedding dimension d , its embedding-table cost is approximately:

$$\text{parameters} = |V|d$$

BPE is frequency-driven, not likelihood-optimal. A frequent pair wins even if a different merge would better reflect linguistic structure.

7. Practical Implementation

```
from collections import Counter

def pair_counts(vocab):
    counts = Counter()
    for symbols, frequency in vocab.items():
        for pair in zip(symbols, symbols[1:]):
            counts[pair] += frequency
    return counts

def merge_pair(vocab, pair):
    merged_vocab = Counter()
    for symbols, frequency in vocab.items():
        output, i = [], 0
        while i < len(symbols):
            if i + 1 < len(symbols) and symbols[i:i + 2] == pair:
                output.append(symbols[i] + symbols[i + 1])
                i += 2
            else:
                output.append(symbols[i])
                i += 1
        merged_vocab[tuple(output)] += frequency
    return merged_vocab

def train_bpe(word_frequencies, num_merges):
    vocab = Counter({tuple(word) + ("</w>"), count
                    for word, count in word_frequencies.items()})
    merges = []
    for _ in range(num_merges):
        counts = pair_counts(vocab)
        if not counts:
            break
        best_pair, _ = counts.most_common(1)[0]
        merges.append(best_pair)
        vocab = merge_pair(vocab, best_pair)
    return merges, vocab

corpus = {"low": 5, "lower": 2, "newest": 6, "widest": 3}
merges, vocabulary = train_bpe(corpus, num_merges=8)

print("Learned merges:", merges)
```



```
for symbols, count in vocabulary.items():  
    print(count, symbols)
```

For production, use the optimized `tokenizers` library and save the complete tokenizer artifact, including normalization, pre-tokenization, merge ranks, vocabulary, and special tokens.

8. Code Explanation

`pair_counts` counts adjacent tuples and weights them by word frequency. `merge_pair` performs a non-overlapping left-to-right replacement. `train_bpe` starts each word as characters plus `</w>`, repeatedly chooses the most frequent pair, and records merge order. Deterministic tie-breaking is important in a production trainer; the compact example relies on `Counter` insertion order.

9. Training / Evaluation

Train on a representative, deduplicated sample rather than only benchmark text. Useful intrinsic measures are fertility (tokens per word), characters/bytes per token, unknown-token rate, vocabulary coverage, and round-trip correctness. Downstream evaluation is decisive: compare task quality, sequence length, latency, and model size while keeping the architecture and data fixed.

Use document-level train/validation separation if tokenizer choices are being tuned against downstream metrics. Inspect domain terms, names, URLs, numbers, code, emojis, and each supported language. A low `[UNK]` rate alone does not imply efficient segmentation.

10. Complexity and Cost

Naively rescanning every word after every merge is expensive: roughly $O(MN)$ for M merges and N corpus symbols, plus pair-counting overhead. Efficient trainers update only affected pair counts using heaps/indexes. Encoding is near-linear for optimized implementations but depends on the merge data structure.

A larger vocabulary shortens sequences but enlarges the input embedding and often the output softmax. Byte-level BPE guarantees coverage but can expand uncommon scripts or malformed byte sequences.

11. Common Use Cases

- Decoder-only language models and text generators
- Code completion, where punctuation and identifiers need exact recovery
- Multilingual models needing open-vocabulary behavior
- Speech/text systems with subword output units
- Domain tokenizers for biomedical or legal corpora

12. Common Mistakes

- Describing BPE tokens as linguistically correct morphemes
- Forgetting to weight pair counts by word frequency
- Applying merges in arbitrary order during encoding
- Training on normalized text but encoding unnormalized text
- Comparing tokenizers only by vocabulary size

- Adding tokens without resizing and training model embeddings
- Losing byte-to-Unicode or special-token configuration when saving

13. Edge Cases / Limitations

BPE uses local frequency, so segmentations can be unintuitive and unstable across corpora. Rare languages may consume far more tokens than English. A token can include leading whitespace, and the same visible string may tokenize differently with different normalization or context. Character BPE may still need [UNK] for unseen characters; byte-level BPE avoids that but may produce many tokens for some Unicode text.

14. Variations

Variation	What changes	When to use	Importance
Character BPE	Base symbols are characters	Controlled alphabets	Placement: high
Byte-level BPE	Base symbols are bytes with reversible mapping	General LLMs, code, arbitrary Unicode	Placement/projects: very high
BPE dropout	Randomly skips merges during training	Tokenization regularization	Research: medium
Vocabulary-restricted BPE	Prevents invalid/rare merged units	Translation and controlled domains	Research: medium
Online/dynamic BPE	Adapts segmentation	Experimental distribution shift	Research: low

15. Related Topics

BPE and WordPiece both build subwords iteratively, but WordPiece uses a likelihood-inspired merge score and commonly performs greedy longest-match tokenization. SentencePiece is a language-independent tokenizer framework that can train BPE or Unigram models directly on raw text. Byte-level BPE connects token coverage to UTF-8. Vocabulary size affects embeddings, softmax cost, context length, and RAG chunk budgets.

16. Interview Questions

1. **What problem does BPE solve?** It balances word-level efficiency with character/byte-level open-vocabulary coverage.
2. **How is the next merge chosen?** The most frequent adjacent symbol pair in the weighted training corpus is merged.
3. **Why store merge order?** Overlapping candidate merges must be applied according to learned ranks for deterministic encoding.
4. **Can BPE represent an unseen word?** Yes, if all of its base symbols are covered; byte-level BPE covers arbitrary byte sequences.
5. **BPE versus WordPiece?** BPE commonly uses raw pair frequency; WordPiece uses a score that discounts frequent individual pieces and uses longest-match inference.
6. **Why use byte-level BPE?** It removes character-level OOV failures and preserves arbitrary text reversibly.

7. **What is tokenizer fertility?** The average number of tokens required per word; high fertility means longer, more expensive sequences.
8. **Does a larger vocabulary always help?** No. It shortens sequences but increases embedding/output parameters and may learn brittle rare tokens.
9. **Is BPE linguistically aware?** No. Morphological-looking units emerge only because of corpus statistics.
10. **What must be versioned with a BPE tokenizer?** Vocabulary, ordered merges, normalizer, pre-tokenizer, byte mapping, and special-token IDs.
11. **How would you evaluate a domain BPE tokenizer?** Measure compression and coverage on held-out domain text, then compare downstream quality and cost.
12. **What happens after adding tokens to a pretrained tokenizer?** The model's embedding/output matrices must be resized and the new rows trained.

17. Practice Tasks

- Extend the implementation with deterministic tie-breaking and an `encode` function.
- Train vocabularies of 2K, 8K, and 32K on the same corpus and plot fertility.
- Compare English, Hindi, Python, URLs, and emoji under GPT-2 byte BPE.
- Find a bug caused by applying merges by frequency rather than stored rank.
- Add BPE dropout and measure whether it improves a small classifier.

18. Project Ideas

Project name	What it does	Tech stack	Dataset suggestion	Resume value
Domain BPE Benchmark	Trains and compares domain vocabularies	Python, HF Tokenizers, PyTorch	arXiv or PubMed abstracts	Shows controlled NLP experimentation
Multilingual Token Cost Auditor	Measures fertility, latency, and fairness across scripts	Python, Transformers, Streamlit	FLORES-200 samples	Strong AI engineering/evaluation story
Code Tokenizer Explorer	Visualizes merges for identifiers and syntax	Tokenizers, FastAPI, React	The Stack subset	Demonstrates tokenizer internals and tooling

19. Quick Revision

- **Key idea:** iteratively merge the most frequent adjacent symbols.
- **Main formula:** $(a^*, b^*) = \arg \max \text{count}(a, b)$.
- **When to use:** open-vocabulary tokenization with good compression.
- **Important metrics:** fertility, bytes/token, round-trip accuracy, downstream quality.
- **Common traps:** treating merges as morphology and losing merge rank/configuration.
- **Interview one-liner:** BPE learns a frequency-ordered compression vocabulary between characters and words.

20. Final Cheat Sheet

Item	Summary
Definition	Greedy subword algorithm that repeatedly merges frequent adjacent symbols
Input/output	Raw text → subword IDs; IDs → recoverable text
Main steps	Base symbols, count pairs, merge best pair, save ranks, encode
Key hyperparameters	Base alphabet, vocabulary size/merge count, min frequency, normalization
Metrics	Tokens/word, bytes/token, OOV rate, latency, downstream score
Pros	Simple, fast, reusable subwords, byte variant has full coverage
Cons	Frequency-only, corpus-sensitive, unequal token cost across languages
Best use cases	General LLMs, code models, domain-specific tokenizers

WordPiece

1. Overview

WordPiece is a subword tokenization method associated with BERT and earlier machine-translation systems. It builds a fixed vocabulary from smaller units and encodes a word using greedy longest-match-first segmentation. In BERT-style tokenizers, continuation pieces are often marked with `##`, as in `playing -> play + ##ing`.

WordPiece controls vocabulary size, reduces unknown words, and lets related surface forms share parameters. It is especially relevant in interviews because BERT's vocabulary, special tokens, and whole-word alignment depend on it.

2. Intuition

Imagine a box of word fragments. For each word, take the longest fragment that matches its beginning, then repeat on the remaining characters. If `un`, `afford`, and `##able` are available, an unseen form can be assembled rather than mapped entirely to `[UNK]`.

Training favors pairs that are useful together relative to how common their individual parts already are. This discourages spending merges only on universally frequent symbols.

3. Prerequisites

- Vocabulary, token IDs, and embeddings
- Conditional probability and frequency counts
- Greedy longest-prefix matching
- BERT input conventions: `[CLS]`, `[SEP]`, `[MASK]`, `[PAD]`
- Text normalization and pre-tokenization

4. Core Concepts

Concept	What it means	Why it matters	Example	Interview angle
Initial vs continuation token	Continuations carry a marker	Preserves word-boundary information	<code>play, ##ing</code>	<code>##</code> is notation, not semantic content
Longest-match-first	Select longest valid prefix repeatedly	Defines deterministic inference	<code>unaffable</code>	Failure of one suffix may yield <code>[UNK]</code>
Merge score	Pair association relative to component frequency	Differs from plain BPE frequency	$f(ab)/(f(a)f(b))$	WordPiece vs BPE
<code>[UNK]</code>	Fallback for an unsegmentable word	Base alphabet may not provide full coverage	unseen script	Why byte tokenizers avoid it
Basic tokenizer	Normalization and word splitting before WordPiece	Changes final tokens	lowercasing/punctuation	<code>BertTokenizer</code> has multiple stages

5. Algorithm / Working Process

Training is commonly described as follows:

1. Normalize and pre-tokenize the corpus into words.
2. Initialize a character inventory; mark noninitial characters as continuations.
3. Count token and adjacent-pair frequencies.
4. score candidate merges using association/likelihood improvement.
5. Add the best merged token and repeat to the target vocabulary size.

Inference for each word:

1. Set a pointer at the word start.
2. Find the longest vocabulary token matching the remaining prefix; use continuation-marked tokens after position zero.
3. Append it and advance the pointer.
4. If no match exists, emit `[UNK]` for the word under the classic algorithm.
5. Add special tokens and convert pieces to IDs.

6. Mathematical Foundation

A commonly presented WordPiece merge score is:

$$\text{score}(a, b) = \frac{f(ab)}{f(a)f(b)}$$

The exact historical/proprietary implementation details are not identical across libraries, so interview answers should emphasize likelihood-inspired association rather than claim one universal trainer formula. The score

prefers a pair that co-occurs disproportionately often, not merely one containing high-frequency components.

Greedy encoding selects:

$$t_i = \arg \max_{t \in V: t \text{ matches prefix}(s_i)} |t|$$

and advances by $|t_i|$. Embedding parameters remain $|V|d$.

7. Practical Implementation

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "Tokenizers make unbelievably rare words manageable."
encoded = tokenizer(text, return_offsets_mapping=True)

tokens = tokenizer.convert_ids_to_tokens(encoded["input_ids"])
for token, token_id, span in zip(
    tokens, encoded["input_ids"], encoded["offset_mapping"]
):
    print(f"{token:15} id={token_id:5} characters={span}")

# Verify the model-facing representation.
assert len(encoded["input_ids"]) == len(encoded["attention_mask"])
print(tokenizer.decode(encoded["input_ids"], skip_special_tokens=True))
```

8. Code Explanation

`AutoTokenizer` loads the vocabulary and the matching normalization rules. `return_offsets_mapping=True` connects every subword to its character span, which is essential for token classification and answer-span tasks. `convert_ids_to_tokens` exposes `##` continuation pieces. Decoding may normalize spacing/case, so offset mappings—not decoded text matching—should drive label alignment.

9. Training / Evaluation

Train on the same language and domain distribution expected downstream. Reserve vocabulary capacity for special tokens and inspect `[UNK]` rates per language, not only globally. Evaluate fertility, continuation-piece rate, sequence lengths, word-boundary recovery, tokenization speed, and downstream scores.

For NER or QA, split documents before constructing token windows to avoid leakage. Track label alignment errors separately from model errors. A tokenizer improvement is meaningful only if it improves quality or cost on held-out data.

10. Complexity and Cost

Naive longest-match inference can try many substrings, but trie-based implementations are close to linear in input length. Training requires iterative candidate scoring and is more expensive than a single corpus pass.

The vocabulary changes both embedding size and sequence length; attention cost then scales approximately as $O(n^2d)$ in sequence length n .

11. Common Use Cases

- BERT-family classification and embedding models
- Named-entity recognition with subword label alignment
- Extractive question answering using offset mappings
- Multilingual encoder models such as multilingual BERT
- Domain-adapted encoders for medicine, finance, or legal text

12. Common Mistakes

- Assuming every `##` token is a true suffix
- Using the slow tokenizer and expecting offset mappings
- Assigning a word label to every subword without an explicit policy
- Forgetting that uncased BERT normalizes case
- Pairing BERT weights with a different WordPiece vocabulary
- Saying WordPiece always has zero OOVs
- Treating vocabulary IDs as meaningful ordinal values

13. Edge Cases / Limitations

Classic WordPiece can map an entire word to `[UNK]` when even one remaining character cannot be matched. Agglutinative languages and unseen scripts may have high fertility. Longest match is locally greedy, not a globally probabilistic segmentation. Normalization can erase case or accents that matter to a task.

14. Variations

Variation	What changes	When to use	Importance
Cased WordPiece	Retains case distinctions	NER and case-sensitive domains	Projects: high
Uncased WordPiece	Lowercases and may strip accents	General English BERT tasks	Placement: high
Whole-word masking	Masks all pieces of a selected word	MLM pretraining	Placement/research: high
Multilingual vocabulary	Shared pieces across languages	Multilingual encoders	Projects: high
Domain vocabulary adaptation	Adds/retrains domain pieces	Heavy terminology shift	Research: medium; test against reuse

15. Related Topics

WordPiece is central to masked language modeling because BERT predicts masked WordPiece IDs. Compared with BPE, its training score is likelihood-inspired rather than raw pair count. Compared with SentencePiece

Unigram, inference is greedy rather than choosing the most probable segmentation. Offset mappings connect WordPiece to NER, QA, and span evaluation.

16. Interview Questions

1. **What is WordPiece?** A fixed-vocabulary subword method with likelihood-inspired vocabulary learning and greedy longest-match encoding.
2. **What does ## mean?** The piece continues a word; it is tokenizer notation, not a linguistic tag.
3. **How does inference work?** Repeatedly select the longest vocabulary item matching the remaining word prefix.
4. **Why can WordPiece produce [UNK]?** Its base inventory may not cover a character or valid continuation.
5. **How does it differ from BPE?** The typical training criterion differs; BPE selects frequent pairs, while WordPiece favors association/likelihood improvement.
6. **Why are offsets useful?** They map subwords back to source character spans for NER and QA.
7. **How do you label split words in NER?** Label the first subword and ignore or consistently transform later pieces, matching the evaluation scheme.
8. **Why must tokenizer and BERT checkpoint match?** Token IDs index checkpoint-specific learned embeddings.
9. **What is whole-word masking?** If a word is selected, all of its WordPiece tokens are masked together.
10. **How does vocabulary size affect BERT?** It trades embedding/softmax parameters against sequence length and fragmentation.
11. **Is longest-match globally optimal?** No; it is a deterministic greedy rule.
12. **How would you detect poor domain tokenization?** Measure high fertility and [UNK] rate, inspect terms, and test downstream quality.

17. Practice Tasks

- Write a longest-prefix encoder using a vocabulary set and continuation marker.
- Align BIO NER labels to BERT offsets and ignore special/subsequent pieces.
- Compare `bert-base-cased` and `bert-base-uncased` on names and acronyms.
- Diagnose a QA model whose answer spans are shifted by normalization.
- Train a small WordPiece tokenizer and compare it with BPE at equal vocabulary size.

18. Project Ideas

Project name	What it does	Tech stack	Dataset suggestion	Resume value
NER Alignment Lab	Visualizes words, subwords, BIO tags, and offsets	Transformers, PyTorch, Streamlit	CoNLL-2003	Shows practical preprocessing expertise
Domain Vocabulary Study	Tests whether retraining vocabulary improves an encoder	HF Tokenizers, Transformers	PubMed or legal judgments	Demonstrates rigorous ablation work
Multilingual Fertility	Audits token cost and [UNK] behavior	Python, Pandas, Plotly	FLORES-200 samples	Strong fairness/evaluation

Project name	What it does	Tech stack	Dataset suggestion	Resume value
Dashboard				signal

19. Quick Revision

- **Key idea:** learn subwords, then greedily take the longest matching piece.
- **Main formula:** score pairs by association, often shown as $f(ab)/(f(a)f(b))$.
- **When to use:** BERT-compatible encoder pipelines.
- **Important metrics:** fertility, [UNK] rate, alignment correctness, downstream F1.
- **Common traps:** mishandling ##, case normalization, or span labels.
- **Interview one-liner:** WordPiece is BERT's subword vocabulary plus longest-match-first segmentation.

20. Final Cheat Sheet

Item	Summary
Definition	Likelihood-inspired subword vocabulary with greedy longest-prefix encoding
Input/output	Pre-tokenized/normalized words → WordPiece IDs and offsets
Main steps	Normalize, split words, longest match, add specials, map IDs
Key hyperparameters	Vocabulary size, min frequency, continuation prefix, casing/normalization
Metrics	[UNK] rate, pieces/word, sequence length, task score
Pros	Compact vocabulary, reusable pieces, mature BERT tooling
Cons	Greedy segmentation, possible [UNK], alignment complexity
Best use cases	BERT-style classification, NER, QA, embeddings

SentencePiece

1. Overview

SentencePiece is a language-independent tokenizer framework that trains directly on raw Unicode text. It treats whitespace as a normal symbol—commonly displayed as `_`—and supports both BPE and Unigram language-model tokenization. Models such as T5, ALBERT, XLNet, and many multilingual or LLaMA-family tokenizers use SentencePiece or closely related configurations.

Its practical value is reproducibility: normalization, segmentation, vocabulary, and decoding behavior are stored in one model artifact. It avoids requiring an English-specific whitespace tokenizer before subword learning.

2. Intuition

Instead of first declaring that spaces separate words, SentencePiece escapes spaces into a visible marker and learns pieces over the whole stream. `Hello world` might become `['_Hello', '_world']`. Because the

leading-space marker belongs to a token, decoding can reconstruct word boundaries consistently.

In Unigram mode, it starts with many candidate pieces and removes pieces that contribute least to corpus likelihood—like pruning a phrase dictionary while preserving good explanations of the text.

3. Prerequisites

- Unicode normalization and UTF-8 text
- Subword tokenization and vocabulary size trade-offs
- Probability, log-likelihood, and dynamic programming
- BPE basics
- Special-token handling and model/tokenizer compatibility

4. Core Concepts

Concept	What it means	Why it matters	Simple example	Common interview angle
Raw-text training	No mandatory language-specific word splitter	Works across scripts/languages	Japanese text	Why it is language independent
<code>_</code> metaspace	Encoded whitespace marker	Supports reversible boundaries	<code>_machine</code>	Leading space changes token identity
Unigram model	Probability distribution over pieces	Allows multiple segmentations	<code>un + able</code> vs <code>u + nable</code>	Unigram vs BPE
BPE mode	Frequent merges within SentencePiece	Offers deterministic BPE behavior	<code>model_type=bpe</code>	Framework vs algorithm distinction
Normalization	Canonicalizes Unicode/text	Controls vocabulary and reversibility	NFKC-like rules	Normalization may remove information
Subword regularization	Samples alternative segmentations	Data augmentation/robustness	different splits per epoch	<code>nbest_size</code> , <code>alpha</code>
Byte fallback	Represents missing characters with byte pieces	Avoids unknowns	unseen Unicode	Useful for general LLMs

5. Algorithm / Working Process

Common preprocessing:

1. Read raw sentences and normalize Unicode according to configuration.
2. Convert spaces to the metaspace symbol.
3. Build required characters and reserve special tokens.

Unigram training:

1. Generate a large seed vocabulary of candidate substrings.
2. Assign each piece an initial probability.
3. Use dynamic programming/EM-style estimation to score possible segmentations.
4. Remove low-utility pieces while preserving required symbols.
5. Repeat until the target vocabulary size is reached.

At inference, Viterbi decoding selects the highest-probability segmentation, or sampling returns alternate segmentations for regularization. BPE mode instead applies learned merges.

6. Mathematical Foundation

For a segmentation $s = (t_1, \dots, t_k)$ under a Unigram model:

$$P(s) = \prod_{i=1}^k p(t_i), \quad \log P(s) = \sum_{i=1}^k \log p(t_i)$$

The sentence probability marginalizes all valid segmentations $\mathcal{S}(x)$:

$$P(x) = \sum_{s \in \mathcal{S}(x)} P(s)$$

Viterbi encoding chooses:

$$s^* = \arg \max_{s \in \mathcal{S}(x)} \sum_{t \in s} \log p(t)$$

Dynamic programming avoids enumerating every segmentation. Subword regularization samples s from a temperature-smoothed distribution rather than always using s^* .

7. Practical Implementation

```
# pip install sentencepiece
from pathlib import Path
from tempfile import TemporaryDirectory
import sentencepiece as spm

sentences = [
    "Transformers process token sequences.",
    "SentencePiece learns directly from raw text.",
    "Subword models handle rare and unseen words.",
] * 50

with TemporaryDirectory() as directory:
    corpus = Path(directory) / "corpus.txt"
    corpus.write_text("\n".join(sentences), encoding="utf-8")
```

```
prefix = str(Path(directory) / "demo")

spm.SentencePieceTrainer.train(
    input=str(corpus),
    model_prefix=prefix,
    model_type="unigram", # change to "bpe" for SentencePiece BPE
    vocab_size=64,
    character_coverage=1.0,
    hard_vocab_limit=False,
    bos_id=1,
    eos_id=2,
)

tokenizer = spm.SentencePieceProcessor(model_file=prefix + ".model")
text = "SentencePiece handles tokenization."
pieces = tokenizer.encode(text, out_type=str)
ids = tokenizer.encode(text, out_type=int)

print(pieces)
print(ids)
assert tokenizer.decode(ids) == text
```

8. Code Explanation

The example builds a temporary UTF-8 corpus and trains a Unigram model. `character_coverage=1.0` preserves all observed characters in this small multilingual-safe demonstration. `hard_vocab_limit=False` allows a smaller realizable vocabulary for the tiny corpus. The `.model` file is the authoritative artifact; the final assertion verifies lossless round-trip behavior for the test sentence.

In a real project, train on millions of representative lines, define PAD/UNK/BOS/EOS IDs explicitly to match the model configuration, and version the `.model` file.

9. Training / Evaluation

Sample by domain and language intentionally; raw web frequency can starve low-resource languages. Deduplicate text and keep validation documents separate. Tune vocabulary size, character coverage, normalization, model type, and byte fallback.

Measure pieces per word/character, byte or character coverage, unknown rate, round-trip fidelity, per-language fertility, tokenization throughput, and downstream quality. For sampled segmentation, compare deterministic evaluation against stochastic training behavior.

10. Complexity and Cost

Unigram training is costlier than greedy encoding because it repeatedly evaluates candidate pieces and expected likelihood. Viterbi encoding is approximately $O(nL)$ for input length n and maximum candidate-piece length L , with tries and pruning improving constants. BPE mode has merge-based costs.

Token sequence length affects Transformer attention quadratically, while vocabulary size affects embedding and output layers linearly. Tokenizer training is usually CPU-heavy but far cheaper than model pretraining.

11. Common Use Cases

- Multilingual and non-whitespace-delimited language models
- T5-style encoder-decoder models
- LLaMA-family and other decoder tokenizers
- Machine translation and speech recognition
- Subword regularization for robustness
- Reproducible domain-specific tokenizer training

12. Common Mistakes

- Calling SentencePiece a single algorithm; it is a framework supporting BPE and Unigram
- Removing `_` or pre-splitting text inconsistently
- Assuming normalization is lossless for every configuration
- Changing special-token IDs after model training
- Choosing low character coverage for a diverse script set
- Reporting global fertility while hiding poor languages
- Saving only a text vocabulary and losing the `.model` configuration

13. Edge Cases / Limitations

Normalization may collapse distinctions important for code, identifiers, accents, or security analysis. Small corpora produce unstable pieces. Unseen characters become `<unk>` unless byte fallback or sufficient character coverage is configured. A shared multilingual vocabulary can allocate capacity unevenly and increase token cost for low-resource scripts.

14. Variations

Variation	What changes	When to use	Importance
SentencePiece Unigram	Probabilistic piece inventory and Viterbi/sampling	Multilingual models, regularization	Placement/research: very high
SentencePiece BPE	Merge-based vocabulary	Deterministic, BPE- compatible pipelines	Placement: high
Character model	Each character is a piece	Very small alphabets/baselines	Projects: medium
Word model	Words are pieces	Closed-vocabulary baselines	Placement: low
Byte fallback	Adds byte pieces for uncovered text	General-purpose and code models	AI engineering: high
Subword regularization	Samples alternative segmentations	Robust training with Unigram/BPE dropout	Research: high

15. Related Topics

SentencePiece BPE is an implementation choice for BPE, while SentencePiece Unigram is a distinct probabilistic algorithm. The `_` marker plays a role similar to leading-space markers in byte-level BPE. Token sampling is a form of input regularization. Special-token IDs connect directly to attention masks, causal generation, padding, and checkpoint compatibility.

16. Interview Questions

1. **What is SentencePiece?** A language-independent tokenizer framework that learns from raw Unicode text and supports Unigram and BPE models.
2. **What does `_` represent?** Encoded whitespace, allowing word boundaries to be part of token pieces.
3. **Why is it language independent?** It does not require a language-specific whitespace/word tokenizer.
4. **Unigram versus BPE?** Unigram prunes a probabilistic candidate inventory and can score multiple segmentations; BPE greedily builds pieces via merges.
5. **What is Viterbi decoding here?** Dynamic programming that finds the maximum-probability piece segmentation.
6. **What is subword regularization?** Sampling alternate valid segmentations during training to reduce dependence on one tokenization.
7. **What does character coverage control?** The fraction of observed characters kept explicitly in the base inventory.
8. **Why use byte fallback?** To represent unseen characters without collapsing them to `<unk>`.
9. **Why must special-token IDs match the model?** Model embeddings and generation logic assign fixed meanings to those IDs.
10. **How do you evaluate a multilingual tokenizer?** Report fertility, coverage, cost, and downstream performance per language.
11. **Can decode differ from the original input?** Yes, normalization or cleanup can make tokenization non-lossless unless configured and tested.
12. **Why save the `.model` artifact?** It contains piece scores and processing configuration, not just token strings.

17. Practice Tasks

- Train BPE and Unigram SentencePiece models at the same vocabulary size.
- Compare deterministic and sampled Unigram segmentations.
- Audit per-language fertility on five scripts.
- Debug a checkpoint whose BOS/EOS IDs do not match its tokenizer.
- Enable byte fallback and test rare Unicode, emoji, and code round trips.

18. Project Ideas

Project name	What it does	Tech stack	Dataset suggestion	Resume value
Multilingual Tokenizer Builder	Trains a balanced shared vocabulary and audits languages	SentencePiece, Python, FLORES tooling	OSCAR samples + FLORES-200	Demonstrates multilingual data engineering

Project name	What it does	Tech stack	Dataset suggestion	Resume value
Segmentation Regularization Study	Tests sampled tokenization on classification	SentencePiece, PyTorch	AG News or IndicGLUE subset	Research-style controlled experiment
Tokenizer Compatibility Checker	Validates special IDs, round trips, and model config	Transformers, SentencePiece, CLI	Public HF checkpoints	Useful production tooling project

19. Quick Revision

- **Key idea:** learn subwords directly from normalized raw text with whitespace encoded as a symbol.
- **Main formula:** Unigram chooses $\arg \max_s \sum_{t \in s} \log p(t)$.
- **When to use:** multilingual/raw-text tokenization or probabilistic segmentation.
- **Important metrics:** per-language fertility, unknown rate, round-trip fidelity, task score.
- **Common traps:** confusing framework with algorithm and mismatching special IDs.
- **Interview one-liner:** SentencePiece packages normalization and BPE/Unigram segmentation into a reproducible raw-text tokenizer.

20. Final Cheat Sheet

Item	Summary
Definition	Raw-text tokenizer framework supporting Unigram, BPE, character, and word models
Input/output	Unicode text $\leftrightarrow$ pieces/IDs with metaspace boundaries
Main steps	Normalize, escape spaces, learn pieces, Viterbi/merge encode, decode
Key hyperparameters	Model type, vocabulary size, coverage, normalization, byte fallback, special IDs
Metrics	Fertility, unknown rate, throughput, round-trip fidelity, downstream quality
Pros	Language independent, reproducible artifact, probabilistic segmentation option
Cons	Configuration-sensitive, normalization risks, multilingual imbalance
Best use cases	Multilingual LLMs, translation, T5/LLaMA-compatible pipelines

Self-Attention

1. Overview

Self-attention lets each token look at other tokens in the same sequence and decide which ones are important. It is the central operation in Transformers. It powers LLMs, BERT classifiers, translation models, summarizers, code models, and multimodal models.

2. Intuition

In the sentence "The animal did not cross the road because it was tired", the model must decide what "it" refers to. Self-attention allows "it" to attend strongly to "animal" instead of unrelated words.

3. Prerequisites

- Vectors and matrix multiplication
- Softmax
- Embeddings
- Gradients and neural networks
- Sequence modeling

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Query Q	What token is looking for	Determines attention request	pronoun asks for referent	Q/K/V roles
Key K	What token offers	Used for matching	noun has matching key	Dot-product score
Value V	Information passed	Weighted and aggregated	noun representation	Output is value mixture
Attention score	Q-K similarity	Chooses important tokens	high score to subject	Scaling by $\sqrt{d_k}$
Mask	Blocks positions	Prevents padding/future access	causal mask	Decoder correctness

5. Algorithm / Working Process

Input: token representations X .

1. Project X into queries, keys, and values.
2. Compute similarity scores QK^T .
3. Scale scores by $\sqrt{d_k}$.
4. Add mask if needed.
5. Apply softmax to get attention weights.
6. Multiply weights by values.
7. Return context-aware token representations.

Output: each token becomes a weighted summary of relevant tokens.

6. Mathematical Foundation

Scaled dot-product attention:


```

Q = XW_Q
K = XW_K
V = XW_V

Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) V

```

Softmax:

```
softmax(z_i) = exp(z_i) / sum_j exp(z_j)
```

The scaling prevents large dot products from saturating softmax when d_k is large.

7. Practical Implementation

```

import math
import torch
import torch.nn.functional as F

def self_attention(x, mask=None):
    batch, seq_len, d_model = x.shape
    wq = torch.randn(d_model, d_model)
    wk = torch.randn(d_model, d_model)
    wv = torch.randn(d_model, d_model)

    q = x @ wq
    k = x @ wk
    v = x @ wv

    scores = q @ k.transpose(-2, -1) / math.sqrt(d_model)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))

    weights = F.softmax(scores, dim=-1)
    return weights @ v, weights

x = torch.randn(2, 5, 32)
out, attn = self_attention(x)
print(out.shape, attn.shape)

```

8. Code Explanation

The code creates query, key, and value projections. `q @ k.transpose(-2, -1)` computes token-to-token similarity. Softmax converts scores into weights. Multiplying by `v` creates the final context vector.

9. Training / Evaluation

Self-attention is trained end-to-end through the model loss. For classification, evaluate accuracy/F1. For language models, evaluate cross-entropy and perplexity. Attention maps can be inspected, but attention weights are not always faithful explanations.

10. Complexity and Cost

For sequence length n and hidden size d :

```
time complexity is approximately  $O(n^2 d)$ 
memory complexity is approximately  $O(n^2)$ 
```

The quadratic term comes from the $n \times n$ attention matrix.

11. Common Use Cases

- Long-range dependency modeling
- Machine translation
- Text classification
- Summarization
- Code generation
- Document question answering

12. Common Mistakes

- Forgetting the $\sqrt{d_k}$ scaling
- Applying softmax over the wrong dimension
- Incorrect mask shape
- Thinking attention weights always explain decisions
- Ignoring quadratic memory cost

13. Edge Cases / Limitations

Self-attention becomes expensive for long sequences. It may over-focus on spurious tokens. Without masks, decoder models can leak future information during training.

14. Variations

Variation	What changes	When to use	Placement value
Bidirectional attention	All tokens attend all tokens	BERT/encoders	Very important
Causal attention	Only previous tokens visible	GPT/decoders	Very important
Cross-attention	Queries attend external sequence	encoder-decoder/RAG-ish fusion	Important
Sparse attention	Limits token pairs	Long context	Advanced

Variation	What changes	When to use	Placement value
FlashAttention	Memory-efficient exact attention	Faster LLM training/inference	Advanced but valuable

15. Related Topics

Self-attention is the base for multi-head attention. It depends on embeddings and positional encoding. It connects to causal language modeling through masks and to encoder-only models through bidirectional attention.

16. Interview Questions

- What is self-attention?
A mechanism where each token computes a weighted summary of tokens in the same sequence.
- What are Q, K, and V?
Query asks, key matches, value provides information.
- Why scale by $\sqrt{d_k}$?
To keep dot-product magnitudes stable for softmax.
- What is causal masking?
Blocking attention to future tokens.
- What is attention complexity?
Roughly $O(n^2 d)$ time and $O(n^2)$ memory.
- Why is self-attention better than RNNs for parallelism?
All token interactions can be computed with matrix operations.
- Does self-attention understand order alone?
No, it needs positional information.
- What is bidirectional attention?
Tokens can attend to both left and right context.
- Why can attention be hard for long documents?
The attention matrix grows quadratically.
- Are attention weights explanations?
They are useful diagnostics but not always faithful explanations.

17. Practice Tasks

- Implement scaled dot-product attention.
- Add a causal mask and verify future tokens receive zero probability.
- Visualize attention weights on a short sentence.
- Compare attention runtime as sequence length doubles.
- Debug a mask broadcasting error.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Tiny Attention Classifier	Uses attention for text classification	PyTorch	AG News	Shows fundamentals

Project	What it does	Tech stack	Dataset	Resume value
Attention Visualizer	Displays token-token weights	Streamlit, PyTorch	Any text	Interview demo
Causal Mask Playground	Demonstrates autoregressive attention	PyTorch	Toy sequences	LLM concept clarity

19. Quick Revision

- Key idea: tokens dynamically weight other tokens.
- Main formula: $\text{softmax}(QK^T / \sqrt{d_k})V$.
- When to use: sequence modeling and Transformers.
- Important metrics: task loss, accuracy, perplexity.
- Common traps: mask errors and quadratic cost.
- Interview one-liner: self-attention lets each token build context from the whole visible sequence.

20. Final Cheat Sheet

Item	Summary
Definition	Token-to-token weighted interaction
Input/output	sequence vectors -> contextual vectors
Main steps	Q/K/V, scores, mask, softmax, weighted sum
Hyperparameters	hidden size, sequence length, mask type
Metrics	loss, perplexity, accuracy
Pros	parallel, captures long dependencies
Cons	quadratic cost
Best use cases	Transformers, LLMs, NLP, code

Multi-Head Attention

1. Overview

Multi-head attention runs several attention operations in parallel, allowing different heads to learn different relationship patterns. One head may track syntax, another entity references, another local context. It is used in nearly every Transformer architecture.

2. Intuition

Instead of one person reading a sentence with one focus, imagine multiple reviewers: one checks grammar, one checks entities, one checks chronology. Their findings are combined.

3. Prerequisites

- Self-attention
- Matrix multiplication
- Tensor shapes
- Concatenation and linear projection
- Transformer architecture

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Head	Independent attention subspace	Learns diverse patterns	syntax head	Head dimension
Head dimension	$d_{\text{model}} / \text{num_heads}$	Controls per-head capacity	$768/12=64$	Divisibility
Concatenation	Join head outputs	Combines information	$[h_1; h_2; \dots]$	Output shape
Output projection	Linear mix after concat	Integrates heads	W_O	Not just stacking
Head redundancy	Some heads may be similar	Pruning possible	unused heads	Interpretability

5. Algorithm / Working Process

1. Project input into Q, K, V.
2. Split Q, K, V into h heads.
3. Run scaled dot-product attention per head.
4. Concatenate head outputs.
5. Apply final output projection.

Output shape usually matches input shape: $(\text{batch}, \text{seq_len}, d_{\text{model}})$.

6. Mathematical Foundation

```
head_i = Attention(XW_i^Q, XW_i^K, XW_i^V)
MultiHead(X) = Concat(head_1, ..., head_h) W^O
```

If $d_{\text{model}} = 768$ and $h = 12$:

```
d_head = d_model / h = 64
```

7. Practical Implementation

```
import torch
import torch.nn as nn

mha = nn.MultiheadAttention(embed_dim=128, num_heads=8, batch_first=True)

x = torch.randn(4, 12, 128) # batch, seq_len, d_model
out, weights = mha(x, x, x, need_weights=True)

print(out.shape)      # [4, 12, 128]
print(weights.shape)  # [4, 12, 12] averaged over heads by default
```

8. Code Explanation

PyTorch's `MultiheadAttention` accepts query, key, and value. For self-attention all three are `x`. `batch_first=True` makes tensors easier to read as batch, sequence, hidden dimension.

9. Training / Evaluation

Multi-head attention is trained end-to-end. Evaluate with the model's task metric. You can inspect per-head attention patterns, ablate heads, or prune heads to study redundancy.

10. Complexity and Cost

Multi-head attention has similar asymptotic complexity to single attention with the same `d_model`:

```
time is approximately  $O(n^2 d_{\text{model}})$ 
memory is approximately  $O(h n^2)$ 
```

More heads can improve expressiveness but increase attention-map memory.

11. Common Use Cases

- Transformer encoder and decoder blocks
- LLM generation
- Translation
- Vision Transformers
- Multimodal attention

12. Common Mistakes

- Choosing `num_heads` that does not divide `d_model`
- Confusing heads with layers
- Assuming every head is interpretable
- Forgetting output projection
- Using too many heads for a tiny model

13. Edge Cases / Limitations

Heads can become redundant. More heads do not guarantee better performance. Attention memory grows with number of heads and sequence length.

14. Variations

Variation	What changes	When to use	Placement value
MHA	Separate Q/K/V per head	Standard Transformer	Very important
MQA	Multiple query heads share K/V	Faster decoding	Important for LLM inference
GQA	Groups of query heads share K/V	Balance quality/speed	Modern LLMs
Cross multi-head attention	Q from decoder, K/V from encoder	Translation/T5	Important
Flash MHA	Efficient kernel	Long/fast training	Advanced

15. Related Topics

Multi-head attention extends self-attention. It connects to decoder-only inference because KV cache size depends on heads. It connects to model cost because attention heads affect memory and speed.

16. Interview Questions

1. Why use multiple heads?
To let the model attend to different patterns in parallel.
2. How is head dimension computed?
 $d_{\text{head}} = d_{\text{model}} / \text{num_heads}$.
3. What happens after heads are computed?
They are concatenated and passed through output projection.
4. Does MHA change output dimension?
Usually no; output returns to d_{model} .
5. MHA vs self-attention?
MHA is several self-attention operations in parallel.
6. What is MQA?
Multi-query attention shares key/value heads to speed decoding.
7. What is GQA?
Grouped-query attention shares K/V within groups.
8. Why can heads be pruned?
Some heads may learn redundant behavior.
9. Why does MHA help language modeling?
Different heads capture syntax, semantics, and long-range dependencies.
10. What is a common shape bug?
Mixing batch-first and sequence-first tensor layouts.

17. Practice Tasks

- Use PyTorch MHA on toy data.
- Print tensor shapes inside a custom MHA implementation.
- Compare 1, 2, 4, and 8 heads on a small classifier.
- Visualize per-head attention.
- Implement causal MHA masking.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Head Visualizer	Shows attention by head	PyTorch, Streamlit	Sentences	Strong explainability demo
Tiny Transformer	Implements MHA from scratch	PyTorch	Tiny Shakespeare	Core LLM skill
MHA Benchmark	Measures speed/memory by heads	PyTorch	Synthetic sequences	Systems awareness

19. Quick Revision

- Key idea: several attention views in parallel.
- Main formula: $\text{Concat}(\text{head}_i)W^O$.
- When to use: standard Transformer blocks.
- Important metrics: loss, speed, memory.
- Common traps: shape errors, overinterpreting heads.
- Interview one-liner: multi-head attention gives the model multiple learned ways to look at the same sequence.

20. Final Cheat Sheet

Item	Summary
Definition	Parallel attention heads
Input/output	sequence vectors -> same-shaped contextual vectors
Main steps	project, split heads, attend, concat, project
Hyperparameters	number of heads, head dimension
Metrics	task score, latency, memory
Pros	richer relationships
Cons	memory and redundancy
Best use cases	all Transformer architectures

Causal Language Modeling

1. Overview

Causal Language Modeling (CLM), also called autoregressive language modeling or next-token prediction, trains a model to predict each token using only earlier tokens. It is the core pretraining objective for GPT, LLaMA, Mistral, and most text-generating LLMs. The causal constraint makes training behavior match generation: at inference, the model repeatedly predicts and appends one next token.

CLM learns grammar, facts, reasoning patterns, code structure, and task behavior from sequences without hand labels. The same objective is also used for supervised instruction tuning, where prompt tokens are context and response tokens are targets.

2. Intuition

Read a sentence through a strip of paper that hides everything to the right. After **The capital of France is**, predict the next token. Move the strip one step and repeat. During training, all positions are evaluated in parallel, but a triangular attention mask prevents every position from peeking at future answers.

It is teacher forcing: when predicting token x_t , the model receives the real prefix $x_{<t}$, not its own earlier sampled mistakes.

3. Prerequisites

- Tokenization, embeddings, and positional information
- Decoder-only Transformer blocks and self-attention
- Softmax, categorical distributions, and cross-entropy
- Attention masks and tensor shapes
- Gradient descent, teacher forcing, and sequence batching
- Decoding methods: greedy, beam, temperature, top- k , and top- p

4. Core Concepts

Concept	What it means	Why it matters	Simple example	Common interview angle
Autoregressive factorization	Joint sequence probability becomes ordered conditionals	Defines generation and loss	$P(x_1, x_2, x_3) = P(x_1)P(x_2 x_1)P(x_3 x_1, x_2)$	
Causal mask	Blocks attention to positions $j > i$	Prevents future-token leakage	upper triangle is $-\infty$	Mask shape and direction
Shifted labels	Logits at t predict token $t + 1$	Aligns outputs with targets	input A B C , targets B C D	Off-by-one bugs
Teacher forcing	Ground-truth prefix is supplied during training	Enables parallel, stable optimization	predict every position at once	Exposure bias

Concept	What it means	Why it matters	Simple example	Common interview angle
BOS/EOS	Sequence boundary tokens	Start context and stop generation	<code><bos> ... <eos></code>	EOS/padding distinction
KV cache	Reuses past attention keys/values during generation	Avoids recomputing the prefix	append one token at a time	Training vs inference cost
Context window	Maximum usable prefix length	Limits dependencies and memory	8K/128K tokens	Position methods and truncation

5. Algorithm / Working Process

Training:

1. Tokenize documents and add boundary tokens according to the model recipe.
2. Pack or pad examples into token sequences.
3. Create labels equal to input IDs; mark ignored padding/prompt positions as `-100` when appropriate.
4. Run decoder blocks with a lower-triangular causal attention mask.
5. Project each hidden state to vocabulary logits.
6. Shift logits/labels by one position and compute token cross-entropy.
7. Backpropagate, update parameters, and monitor validation loss/perplexity.

Inference:

1. Encode a prompt and compute its hidden states/KV cache.
2. Convert final-position logits to a probability distribution.
3. Select or sample a token.
4. Append it and reuse cached keys/values.
5. Stop at EOS, a stopping rule, or the token budget.

Input: token prefix. Output: a distribution over the next token at every position during training, or one next-token distribution during generation.

6. Mathematical Foundation

Autoregressive factorization:

$$P_{\theta}(x_{1:T}) = \prod_{t=1}^T P_{\theta}(x_t \mid x_{<t})$$

Negative log-likelihood over a corpus is:

$$\mathcal{L}_{CLM} = -\frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \log P_{\theta}(x_t^{(n)} | x_{<t}^{(n)})$$

For vocabulary logits z_t , the predicted probability is:

$$P_{\theta}(x_{t+1} = v | x_{\leq t}) = \frac{e^{z_{t,v}}}{\sum_{u \in V} e^{z_{t,u}}}$$

The attention mask is:

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

Perplexity is the exponentiated mean token loss:

$$\text{PPL} = \exp(\mathcal{L}_{\text{token}})$$

Perplexities are comparable only when tokenization and evaluation protocol match.

7. Practical Implementation

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

checkpoint = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint)

# GPT-2 has no dedicated pad token; using EOS is acceptable when labels for
# padded positions are masked and an attention mask is supplied.
tokenizer.pad_token = tokenizer.eos_token
batch = tokenizer(
    ["Attention lets a token", "Language models predict"],
    padding=True,
    return_tensors="pt",
)

labels = batch["input_ids"].clone()
labels[batch["attention_mask"] == 0] = -100 # ignored by cross-entropy

outputs = model(**batch, labels=labels)
print("loss:", outputs.loss.item())
print("perplexity:", torch.exp(outputs.loss).item())

# The model internally shifts logits and labels for causal next-token loss.
with torch.no_grad():
    generated = model.generate(
        **tokenizer("Transformers are", return_tensors="pt"),
        max_new_tokens=20,
        do_sample=True,
        temperature=0.8,
        top_p=0.9,
```

```
        pad_token_id=tokenizer.eos_token_id,  
    )  
    print(tokenizer.decode(generated[0], skip_special_tokens=True))
```

8. Code Explanation

The tokenizer creates `input_ids` and a padding mask. Labels initially copy the inputs because each token is the target for its previous position; Hugging Face causal-LM classes perform the one-token shift internally. Setting padding labels to `-100` prevents them from contributing to PyTorch cross-entropy.

`generate` performs autoregressive inference. `temperature=0.8` rescales logits and `top_p=0.9` samples from the smallest high-probability nucleus. These decoding parameters change output behavior, not the model's learned probabilities.

9. Training / Evaluation

Prepare high-quality, deduplicated, licensed text; remove benchmark contamination and sensitive data. Split at document or source level before chunking. Packing documents improves utilization, but boundary/EOS handling must prevent accidental semantic continuation. For instruction tuning, usually mask prompt/padding labels so loss is computed only on the assistant response.

Track validation negative log-likelihood, token accuracy as a diagnostic, and perplexity with a fixed tokenizer. Generation quality requires task metrics, human preference, factuality, toxicity/safety, and exact-match or pass@k for code. Watch the train/validation loss gap, memorization probes, and source-domain breakdowns.

Important hyperparameters include context length, global token batch size, learning rate/warmup, weight decay, optimizer, precision, gradient clipping, dropout, and data mixture weights. Scaling batch by sequences instead of non-padding tokens can make optimization inconsistent.

10. Complexity and Cost

For dense self-attention with sequence length n , hidden width d , and L layers, training attention is approximately $O(Ln^2d)$ time and $O(Ln^2)$ attention-memory before memory-efficient kernels. Feed-forward layers contribute roughly $O(Lnd^2)$. Training also stores activations and optimizer states.

Without a KV cache, generating T tokens repeatedly recomputes the prefix. With caching, each new token attends to cached keys/values, substantially reducing computation, while cache memory grows approximately $O(LTd)$. Output softmax scales with vocabulary size. Large-scale pretraining requires distributed GPUs/TPUs; small fine-tunes can use PEFT and quantization.

11. Common Use Cases

- Open-ended text and dialogue generation
- Code completion and program synthesis
- Instruction-following assistants after post-training
- Summarization, translation, and extraction via prompting/fine-tuning
- Synthetic data generation
- Autocomplete and structured-data generation

12. Common Mistakes

- Forgetting the one-position label shift or shifting twice
- Allowing tokens to attend to future positions during training
- Including pad tokens in the loss
- Computing instruction loss on prompt tokens unintentionally
- Comparing perplexity across different tokenizers
- Evaluating only teacher-forced loss and ignoring free-running generation
- Using `max_length` when `max_new_tokens` expresses the intended generation budget
- Treating sampling temperature as a training hyperparameter
- Packing documents without EOS/boundary handling

13. Edge Cases / Limitations

CLMs can hallucinate because next-token likelihood is not a truth objective. Teacher forcing creates exposure bias: inference prefixes contain model-generated errors not seen in the same way during training. Finite context prevents direct access to distant information. Repeated decoding is sequential, making generation latency harder to parallelize than training. Rare events, exact arithmetic, fresh facts, and strict constrained outputs may require tools, retrieval, verification, or constrained decoding.

14. Variations

Variation	What changes	When to use	Importance
Standard decoder-only CLM	Full causal attention over prefix	General generation	Placement: essential
Prefix LM	Prefix attends bidirectionally; target remains causal	Conditional generation	Research/projects: medium
Fill-in-the-middle	Predicts missing middle using rearranged context	Code completion/editing	AI engineering: high
Multi-query/grouped-query attention	Shares K/V heads	Faster, smaller KV cache	Modern LLM interviews: high
Sliding-window attention	Restricts local attention span	Long sequences with bounded cost	Projects/research: high
Speculative decoding	Draft model proposes tokens, target verifies	Lower inference latency without changing distribution	Systems interviews: high

15. Related Topics

CLM contrasts with Masked Language Modeling: CLM is unidirectional and naturally generates, while MLM uses bidirectional context and predicts corrupted positions. Causal masks are implemented inside self-attention. Instruction tuning uses the same next-token loss on formatted prompt-response sequences. RLHF

and DPO further adjust the causal policy using preferences. KV caching, quantization, FlashAttention, and speculative decoding reduce deployment cost.

16. Interview Questions

1. **What is causal language modeling?** Predicting each token conditioned only on earlier tokens.
2. **Why is it called causal?** Information flows from past to future; position t cannot use future positions.
3. **How can training predict all positions in parallel?** A triangular mask blocks future attention while matrix operations process all queries simultaneously.
4. **Why shift labels?** Hidden state h_t represents prefix $x_{\leq t}$ and must predict x_{t+1} .
5. **What is teacher forcing?** Training conditions on true preceding tokens rather than sampled model outputs.
6. **What is exposure bias?** At inference the model conditions on its own possibly erroneous tokens, a distribution different from teacher-forced training prefixes.
7. **What does perplexity measure?** Exponentiated average negative log-likelihood per evaluated token; lower is better under the same tokenizer/protocol.
8. **Why mask padding labels with -100?** PyTorch cross-entropy ignores that target value, preventing artificial loss.
9. **Why use a KV cache?** It reuses past keys and values so the model does not recompute the full prefix at every decoding step.
10. **CLM versus MLM?** CLM predicts the next token from left context and supports generation; MLM reconstructs masked tokens using both sides and excels at representations.
11. **Does lower perplexity guarantee better chat quality?** No. It measures token prediction, not instruction following, factuality, safety, or preference.
12. **Why is generation sequential?** Each next-token distribution depends on the token selected immediately before it.

17. Practice Tasks

- Implement a lower-triangular attention mask and verify future weights are zero.
- Fine-tune a tiny causal LM on response-only labels and compare with full-sequence loss.
- Measure validation perplexity with and without document packing.
- Debug an off-by-one loss by manually comparing logits at one position to its label.
- Benchmark greedy, top- k , top- p , and temperature sampling on fixed prompts.

18. Project Ideas

Project name	What it does	Tech stack	Dataset suggestion	Resume value
Tiny Domain GPT	Pretrains/fine-tunes a small decoder and documents scaling/eval	PyTorch, Transformers, W&B/MLflow	TinyStories or domain text	End-to-end LLM training evidence
Code Completion Evaluator	Measures pass@k, latency, and sampling trade-offs	Transformers, Docker sandbox	HumanEval/MBPP	Strong applied LLM systems project

Project name	What it does	Tech stack	Dataset suggestion	Resume value
Prompt-Loss Debugger	Visualizes per-token loss and ignored labels	PyTorch, Gradio/Streamlit	Instruction dataset subset	Shows deep training-pipeline understanding

19. Quick Revision

- **Key idea:** predict the next token using only the prefix.
- **Main formula:** $P(x_{1:T}) = \prod_t P(x_t | x_{<t})$.
- **When to use:** generative language/code models.
- **Important metrics:** validation NLL/perplexity plus task and human generation evaluation.
- **Common traps:** causal-mask direction, double shifting, pad/prompt loss, tokenizer-dependent PPL.
- **Interview one-liner:** CLM trains a decoder with teacher-forced next-token cross-entropy and generates autoregressively.

20. Final Cheat Sheet

Item	Summary
Definition	Autoregressive next-token prediction under a causal attention mask
Input/output	Prefix token IDs → distribution over next vocabulary token
Main steps	Tokenize, causal decoder, vocabulary logits, shifted CE, autoregressive decode
Key hyperparameters	Context length, token batch size, LR, warmup, precision, decoding temperature/top- p
Metrics	NLL, perplexity, task score, win rate, safety/factuality tests, latency
Pros	Self-supervised, scalable, natural generation objective
Cons	Sequential decoding, hallucination, exposure bias, context limit
Best use cases	Chat, code, completion, instruction-following generators

Masked Language Modeling

1. Overview

Masked Language Modeling (MLM) corrupts selected input tokens and trains a model to reconstruct their original identities using context on both sides. BERT, RoBERTa, and many encoder-only models use MLM pretraining to learn bidirectional representations for classification, retrieval, NER, extractive QA, and semantic similarity.

MLM is a denoising objective rather than a natural left-to-right generation process. Its strength is representation learning: every unmasked token can attend to the entire visible sentence.

2. Intuition

It resembles a fill-in-the-blank exercise: The cat [MASK] on the mat. The model uses both cat on the left and on the mat on the right to infer sat. Repeating this over massive unlabeled corpora teaches syntax and semantics without manual labels.

Only corrupted positions provide the main prediction targets, so the model cannot win by simply copying every visible token.

3. Prerequisites

- Tokenization and special tokens such as [MASK]
- Transformer encoders and bidirectional self-attention
- Softmax and multiclass cross-entropy
- Random sampling and probability distributions
- Fine-tuning for classification, NER, retrieval, and QA

4. Core Concepts

Concept	What it means	Why it matters	Simple example	Common interview angle
Masking probability	Fraction selected for prediction	Controls signal and corruption	often 15%	Why not mask everything?
80/10/10 rule	Of selected BERT tokens: mask/random/unchanged	Reduces [MASK] train-test mismatch	80% [MASK]	Selected unchanged tokens still have loss
Dynamic masking	New corruption pattern each epoch/batch	Exposes more prediction targets	RoBERTa collator	Static vs dynamic masking
Bidirectional attention	Tokens use visible left and right context	Produces strong contextual embeddings	fill middle word	Why not directly autoregressive
Loss mask	Loss computed only on selected positions	Prevents trivial identity learning	other labels -100	Attention mask is different
Whole-word masking	All subwords of a chosen word are selected	Avoids leaking word pieces	play, ##ing both masked	Requires word boundaries
Span masking	Masks contiguous spans	Better phrase/structure corruption	mask three adjacent tokens	Connects to T5/span denoising

5. Algorithm / Working Process

1. Tokenize a document and add required special tokens.
2. Choose eligible token positions independently or by word/span, excluding padding and special tokens.

3. In original BERT, for each selected position: replace 80% with `[MASK]`, 10% with a random vocabulary token, and leave 10% unchanged.
4. Feed the corrupted sequence to a bidirectional Transformer encoder.
5. Project hidden states to vocabulary logits.
6. Compute cross-entropy only at selected positions against original token IDs.
7. Update parameters; resample corruption dynamically if configured.

At downstream inference, the MLM head may be used for fill-mask tasks, but usually it is discarded and the encoder is fine-tuned or used for embeddings.

6. Mathematical Foundation

Let M be selected positions and $\tilde{x}$ the corrupted sequence. The objective is:

$$\mathcal{L}_{MLM} = -\frac{1}{|M|} \sum_{i \in M} \log P_{\theta}(x_i \mid \tilde{x})$$

For logits $z_i \in \mathbb{R}^{|V|}$:

$$P_{\theta}(x_i = v \mid \tilde{x}) = \frac{e^{z_{i,v}}}{\sum_{u \in V} e^{z_{i,u}}}$$

Unlike CLM, this does not define a straightforward normalized left-to-right sequence likelihood because predictions are conditionally reconstructed under artificial corruption. Therefore MLM loss and CLM perplexity are not directly comparable. Pseudo-log-likelihood can mask one token at a time, but it is expensive and must be named explicitly.

Expected predicted tokens per sequence are approximately $p_m n$ for masking rate p_m and n eligible positions, so MLM receives fewer supervised target positions per input than CLM.

7. Practical Implementation

```
import torch
from transformers import (
    AutoModelForMaskedLM,
    AutoTokenizer,
    DataCollatorForLanguageModeling,
)

checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForMaskedLM.from_pretrained(checkpoint)

texts = [
    "Transformers build contextual representations.",
    "Masked language models use both left and right context.",
]
examples = [tokenizer(text) for text in texts]

# Dynamic masking: creates corrupted input_ids and labels on every call.
collator = DataCollatorForLanguageModeling(
```

```

        tokenizer=tokenizer,
        mlm=True,
        mlm_probability=0.15,
    )
    batch = collator(examples)

    outputs = model(**batch)
    print("MLM loss:", outputs.loss.item())

    # Verify that only selected positions contribute to loss.
    selected = batch["labels"] != -100
    assert selected.any()
    print("selected targets:", int(selected.sum()))

    # Direct fill-mask inference.
    sentence = "Paris is the [MASK] of France."
    inputs = tokenizer(sentence, return_tensors="pt")
    with torch.no_grad():
        logits = model(**inputs).logits
    mask_index = (inputs["input_ids"] == tokenizer.mask_token_id).nonzero()[0, 1]
    top_ids = logits[0, mask_index].topk(5).indices
    print(tokenizer.convert_ids_to_tokens(top_ids))

```

8. Code Explanation

The collator pads examples, selects eligible tokens, applies the model's masking policy, and creates labels. Unselected positions have label `-100`, so cross-entropy ignores them. Calling the collator again produces a different corruption pattern, which is dynamic masking.

For fill-mask inference, the code locates `[MASK]`, extracts that position's vocabulary logits, and returns the five highest-scoring tokens. In downstream classification, use `AutoModelForSequenceClassification` rather than manually using the MLM logits.

9. Training / Evaluation

Clean and deduplicate the corpus, then split by document/source before creating chunks. Maintain punctuation, casing, and domains appropriate to the intended checkpoint. Dynamic masking increases target coverage across epochs. For whole-word or span masking, test label construction carefully with subword boundaries.

Track validation MLM loss and masked-token accuracy, but judge the encoder on downstream tasks: accuracy/F1 for classification, entity F1 for NER, exact match/F1 for QA, and Recall@k/MRR/nDCG for retrieval. Probe by domain and sequence length. Overfitting appears as improving train reconstruction with stagnant validation/downstream quality.

Key hyperparameters are masking rate/policy, sequence length, batch size in tokens, learning rate, warmup, weight decay, corpus mixture, and number of steps. Domain-adaptive pretraining usually needs a low learning rate and general-domain regression checks.

10. Complexity and Cost

Dense encoder self-attention costs approximately $O(Ln^2d)$ time and $O(Ln^2)$ attention storage. Unlike autoregressive generation, MLM processes a fixed sequence in parallel and normally needs no KV cache. Only about 15% of tokens produce direct vocabulary loss under standard BERT, which is less sample-efficient per processed token than objectives predicting more positions.

Fine-tuned encoder inference typically requires one forward pass and is faster than generating an answer token by token. Vocabulary projection during pretraining has $O(n|V|d)$ naive cost, though only selected positions need the MLM head conceptually.

11. Common Use Cases

- Text classification and intent detection
- Named-entity recognition and sequence labeling
- Extractive question answering
- Reranking and bidirectional text encoders
- Domain-adaptive pretraining for medical/legal/financial text
- Fill-mask analysis and linguistic probing

12. Common Mistakes

- Computing loss on every position instead of selected tokens
- Masking `[CLS]`, `[SEP]`, `[PAD]`, or other protected tokens
- Confusing the padding attention mask with the MLM loss mask
- Treating the 80/10/10 policy as 80% of all input tokens
- Reporting ordinary perplexity as if MLM defined left-to-right likelihood
- Applying independent subword masking when whole-word behavior was intended
- Pretraining on validation/test documents and causing leakage
- Expecting a vanilla encoder MLM to generate fluent long-form text

13. Edge Cases / Limitations

The `[MASK]` token is usually absent at downstream inference, creating a pretrain–fine-tune mismatch. Only selected positions produce direct loss, reducing prediction density. Independent token masking can leak answers through neighboring pieces of the same word. MLM is not naturally suited to free-form autoregressive generation. Very high masking makes reconstruction ambiguous; very low masking provides little learning signal.

14. Variations

Variation	What changes	When to use	Importance
Static masking	Corruptions generated once	Reproducibility/legacy BERT data	Placement: medium
Dynamic masking	Resamples corruption during training	Most modern MLM pipelines	Placement/projects: high
Whole-word masking	Selects every subword of a word	Word-level semantics, WordPiece models	Placement: high

Variation	What changes	When to use	Importance
Span masking	Corrupts contiguous token spans	Phrase/structure learning	Research: high
ELECTRA replaced-token detection	Discriminator classifies replaced tokens	More training signal per token	Placement/research: high
T5 span corruption	Replaces spans with sentinels and generates them	Encoder-decoder pretraining	Placement: high
Permuted LM	Predicts under sampled factorization orders	XLNet-style bidirectional context without [MASK]	Research: medium

15. Related Topics

MLM uses bidirectional self-attention, whereas CLM uses a causal mask. WordPiece affects masking because one word may span multiple tokens. ELECTRA changes reconstruction into replaced-token discrimination. T5 span corruption uses an encoder-decoder and generates missing spans. Domain-adaptive pretraining continues MLM on in-domain unlabeled text before task fine-tuning.

16. Interview Questions

1. **What is masked language modeling?** Reconstructing selected original tokens from a corrupted sequence using bidirectional context.
2. **What is BERT's 80/10/10 rule?** Among the 15% selected tokens, 80% become [MASK], 10% random tokens, and 10% remain unchanged; all selected positions are prediction targets.
3. **Why leave some selected tokens unchanged?** It reduces reliance on [MASK] and trains useful representations for real visible tokens.
4. **Why use random replacements?** They force the model to detect and correct plausible corruption rather than only react to [MASK].
5. **What is dynamic masking?** Sampling new masked positions when examples are collated or revisited.
6. **Why is MLM bidirectional?** There is no causal mask, so a position can attend to visible tokens on both sides.
7. **Why is MLM not ideal for generation?** It does not train a natural left-to-right factorization or stopping process.
8. **Can MLM use perplexity?** Standard CLM perplexity is not directly defined; use MLM loss/accuracy or explicitly computed pseudo-perplexity.
9. **What is whole-word masking?** Selecting all subword pieces belonging to a chosen word.
10. **MLM versus ELECTRA?** MLM predicts original token IDs at selected sites; ELECTRA labels every position as original or generator-replaced.
11. **What does label -100 do?** It marks positions ignored by PyTorch cross-entropy.
12. **How do you evaluate pretrained MLM quality?** Combine held-out reconstruction diagnostics with downstream task/retrieval performance.

17. Practice Tasks

- Implement the 80/10/10 corruption policy while protecting special tokens.
- Compare static and dynamic masking over multiple epochs.
- Continue BERT pretraining on a domain corpus and measure downstream improvement.
- Debug a collator that accidentally includes padding in the MLM loss.
- Compare token masking, whole-word masking, and span masking at equal compute.

18. Project Ideas

Project name	What it does	Tech stack	Dataset suggestion	Resume value
Domain-Adaptive BERT	Continues MLM then evaluates domain classification/NER	Transformers, PyTorch, MLflow	PubMed, SEC filings, or legal text	Strong research + production story
Masking Policy Benchmark	Compares token, whole-word, and span corruption	HF Datasets, Transformers	Wikipedia subset	Demonstrates clean experimental design
MLM Error Explorer	Shows masked predictions by POS/domain/confidence	FastAPI, Transformers, Plotly	WikiText + custom domain	Useful interpretability tooling

19. Quick Revision

- **Key idea:** reconstruct corrupted tokens using both left and right context.
- **Main formula:** $-\sum_{i \in M} \log P(x_i | \tilde{x})$.
- **When to use:** encoder representation pretraining and domain adaptation.
- **Important metrics:** MLM loss/accuracy and downstream F1, Recall@k, or task score.
- **Common traps:** wrong loss mask, masking special tokens, misleading perplexity claims.
- **Interview one-liner:** MLM is bidirectional denoising pretraining; it builds representations rather than a native left-to-right generator.

20. Final Cheat Sheet

Item	Summary
Definition	Predict selected original tokens from a corrupted bidirectional context
Input/output	Corrupted token sequence $\rightarrow$ vocabulary distribution at selected positions
Main steps	Select tokens, corrupt, encode bidirectionally, compute selected-token CE
Key hyperparameters	Mask probability/policy, sequence length, LR, data mix, dynamic/whole-word/span mode
Metrics	Validation MLM loss, masked accuracy, downstream task/retrieval score
Pros	Strong contextual encoders, unlabeled pretraining, parallel inference
Cons	[MASK] mismatch, sparse targets, not natural free-form generation

Item	Summary
Best use cases	Classification, NER, QA, reranking, domain-adaptive encoders

Instruction Tuning

1. Overview

Instruction tuning fine-tunes a pretrained language model on instruction-response examples so it learns to follow user requests. It is a key step that turns a base language model into a useful assistant. It is used in chatbots, coding assistants, enterprise copilots, tool-using agents, and safety-aligned systems.

2. Intuition

A base model predicts likely text. An instruction-tuned model learns the pattern: user asks a task, assistant gives a helpful answer. It is the difference between autocomplete and assistant behavior.

3. Prerequisites

- Pretraining
- Fine-tuning
- Prompt-response datasets
- Supervised learning
- Evaluation of generated outputs

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Instruction dataset	prompts + ideal responses	teaches following	"Summarize..." -> answer	Data quality
SFT	supervised fine-tuning	common tuning stage	train on responses	Loss masking
Chat template	formats roles	model learns dialogue	system/user/assistant	Token format matters
Preference tuning	learns preferred responses	alignment	RLHF/DPO	After SFT
Safety behavior	refusal/helpfulness rules	deployment quality	harmful request handling	Alignment

5. Algorithm / Working Process

1. Start with pretrained base model.
2. Collect instruction-response pairs.

3. Format examples with the model's chat template.
4. Train model to predict assistant response tokens.
5. Validate helpfulness, correctness, and format.
6. Optionally apply preference tuning such as RLHF or DPO.
7. Deploy with system prompts and safety checks.

6. Mathematical Foundation

Supervised instruction tuning loss:

$$L_{\text{SFT}} = - \sum_{t \text{ in assistant response}} \log P(y_t \mid \text{instruction}, y_{<t})$$

Often, loss is masked on user prompt tokens so the model is optimized mainly for assistant responses.

Preference tuning idea:

preferred response should receive higher score/probability than rejected response

DPO-style objective conceptually increases:

$$\log P(y_{\text{chosen}} \mid x) - \log P(y_{\text{rejected}} \mid x)$$

7. Practical Implementation

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("HuggingFaceH4/zephyr-7b-beta")

messages = [
    {"role": "system", "content": "You are a concise ML interview mentor."},
    {"role": "user", "content": "Explain self-attention in two sentences."},
    {"role": "assistant", "content": "Self-attention lets each token weigh other visible tokens. It computes softmax(QK^T/sqrt(d_k))V to build context-aware representations."},
]

formatted = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=False,
)

print(formatted)
```

8. Code Explanation

Instruction-tuned chat models expect a specific role format. `apply_chat_template` formats system, user, and assistant messages exactly as the model was trained to read them.

9. Training / Evaluation

Evaluate with held-out instruction sets, human preference, factuality checks, format adherence, safety tests, and task-specific benchmarks. Watch for overfitting to response style, verbosity, refusal errors, and benchmark contamination.

10. Complexity and Cost

Instruction tuning is cheaper than pretraining but can still be expensive for large models. PEFT methods such as LoRA/QLoRA reduce memory. Data quality matters more than raw dataset size.

11. Common Use Cases

- Chat assistants
- Coding copilots
- Customer support bots
- Tool-using agents
- Structured extraction assistants
- Domain expert assistants

12. Common Mistakes

- Training on low-quality synthetic answers without filtering
- Not masking prompt tokens when intended
- Using wrong chat template
- Confusing instruction tuning with prompting
- Ignoring safety and refusal evaluation
- Over-tuning until general ability drops

13. Edge Cases / Limitations

Instruction tuning improves following instructions but does not guarantee truth. It can make models overly agreeable, verbose, or refusal-prone. It may not add reliable new knowledge.

14. Variations

Variation	What changes	When to use	Placement value
SFT	train on prompt-answer pairs	base assistant behavior	Very important
Multi-turn tuning	dialogue examples	chat assistants	Important
Tool instruction tuning	tool call traces	agents	AI engineering
RLHF	reward model + RL	preference alignment	Important concept

Variation	What changes	When to use	Placement value
DPO	direct preference optimization	simpler preference tuning	Very important

15. Related Topics

Instruction tuning connects pretraining, fine-tuning, prompting, preference alignment, RLHF, DPO, safety evaluation, and chat templates.

16. Interview Questions

1. What is instruction tuning?
Fine-tuning a model on instruction-response examples.
2. Why is it needed?
Base LMs predict text; instruction tuning teaches task-following behavior.
3. SFT meaning?
Supervised fine-tuning.
4. What is a chat template?
The role/message formatting expected by a chat model.
5. Why mask user prompt loss?
To train the model mainly on assistant response behavior.
6. Instruction tuning vs prompting?
Tuning changes weights; prompting changes input.
7. Instruction tuning vs pretraining?
Pretraining learns broad language; instruction tuning aligns behavior to requests.
8. What is RLHF?
Reinforcement learning from human feedback for preference alignment.
9. What is DPO?
A preference tuning method that directly optimizes chosen over rejected responses.
10. Does instruction tuning guarantee factuality?
No, retrieval/evaluation may still be needed.

17. Practice Tasks

- Format examples with a chat template.
- Build a tiny instruction dataset.
- Fine-tune a small model with LoRA for response style.
- Evaluate format-following accuracy.
- Compare base vs instruction-tuned model outputs.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
ML Interview Tutor	Instruction-tuned mentor responses	PEFT, HF	Custom Q&A	Direct placement value
Tool-Calling Assistant Dataset	Creates tool-use examples	Python, JSONL	Synthetic tasks	Agent engineering

Project	What it does	Tech stack	Dataset	Resume value
Preference Pair Evaluator	Compares chosen/rejected answers	DPO/TRL	HH-RLHF style data	Alignment knowledge

19. Quick Revision

- Key idea: teach a base model to follow instructions.
- Main formula: response-token cross-entropy.
- When to use: assistant behavior and task formatting.
- Important metrics: helpfulness, correctness, format adherence, safety.
- Common traps: wrong chat template, low-quality data.
- Interview one-liner: instruction tuning turns a language model into an instruction-following assistant.

20. Final Cheat Sheet

Item	Summary
Definition	Fine-tuning on instruction-response data
Input/output	instruction dataset -> assistant model
Main steps	format, mask, train, validate, align
Hyperparameters	LR, epochs, LoRA rank, max length
Metrics	helpfulness, win rate, safety, format validity
Pros	better task following
Cons	data quality sensitive, no truth guarantee
Best use cases	chatbots, copilots, agents

RLHF

1. Overview

Reinforcement Learning from Human Feedback (RLHF) aligns a pretrained or instruction-tuned language model with human preferences. It is useful when "correctness" is hard to express with a single supervised label, such as helpfulness, harmlessness, tone, refusal behavior, and response quality. Real systems use RLHF or RLHF-like post-training to make chatbots, coding assistants, tutoring agents, summarizers, and customer-support models more useful and safer.

2. Intuition

First teach the model to answer with examples, then ask humans which of two answers is better, then train the model to prefer answers that humans would choose. Analogy: supervised fine-tuning teaches a student by showing solved answers; RLHF is like giving grades and comments, then making the student optimize for better grades.

3. Prerequisites

Transformers, language-model loss, supervised fine-tuning, policy gradients, reward models, KL divergence, preference datasets, PyTorch/Hugging Face basics.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Policy model	The LLM being optimized	Generates candidate responses	Chat model	Why call an LLM a policy?
Preference data	Human-ranked outputs	Captures subjective quality	A preferred over B	Pairwise ranking loss
Reward model	Predicts human preference score	Replaces expensive human scoring during RL	$r_\phi(x, y)$	Reward hacking risk
PPO	Stable RL optimizer often used in RLHF	Updates policy without drifting too far	KL penalty	Why KL to reference model?
Reference model	Frozen copy of SFT model	Prevents collapse and weird outputs	π_{ref}	Alignment vs capability tradeoff

5. Algorithm / Working Process

1. Start with a pretrained LLM.
2. Supervised fine-tune it on high-quality instruction-response data.
3. Collect prompts and multiple model responses.
4. Ask humans or labelers to rank responses.
5. Train a reward model to predict the preferred response.
6. Optimize the policy model with RL, commonly PPO, using reward plus KL penalty.
7. Evaluate with human preference, safety tests, factuality checks, and regression tests.

Input: prompt. Processing: policy generates response, reward model scores it, PPO updates model. Output: aligned chat model.

6. Mathematical Foundation

Preference reward model commonly uses Bradley-Terry loss:

$$P(y_w \succ y_l | x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

$$L_{RM} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

RLHF objective:

$$\max_{\theta} E_{y \sim \pi_\theta(\cdot | x)} [r_\phi(x, y)] - \beta KL(\pi_\theta(\cdot | x) || \pi_{ref}(\cdot | x))$$

The KL term keeps the optimized model close to the original instruction-tuned model.

7. Practical Implementation

```
# Minimal preference reward-model loss in PyTorch
import torch
import torch.nn.functional as F

def reward_model_loss(chosen_rewards, rejected_rewards):
    # chosen_rewards, rejected_rewards: shape [batch]
    return -F.logsigmoid(chosen_rewards - rejected_rewards).mean()

chosen = torch.tensor([3.0, 1.2, 2.1])
rejected = torch.tensor([1.0, 1.5, 0.4])
loss = reward_model_loss(chosen, rejected)
print(loss.item())
```

8. Code Explanation

`chosen_rewards - rejected_rewards` should be positive when the reward model agrees with human labels. `logsigmoid` creates the pairwise ranking loss. The mean gives one scalar training objective.

9. Training / Evaluation

Prepare prompts with chosen/rejected responses. Split by prompt, not by individual response, to avoid leakage. Evaluate reward-model accuracy on held-out pairs, then evaluate final LLM using preference win rate, safety tests, factuality, refusal precision/recall, and task benchmarks. Watch for over-optimization: reward goes up while real response quality drops.

10. Complexity and Cost

RLHF is expensive: it needs human labels, reward-model training, and repeated LLM generation during RL. Memory cost is high because PPO may keep policy, reference, reward, and value models. Inference cost after training is usually the same as the base model.

11. Common Use Cases

Chat assistants, coding assistants, content moderation, summarization, tutoring, medical-style triage assistants with strict safety boundaries, enterprise support bots.

12. Common Mistakes

Using noisy preference labels, optimizing reward too aggressively, ignoring KL, evaluating only with the reward model, mixing train/test prompts, missing safety evals, treating RLHF as a factuality solution, failing to check distribution shift.

13. Edge Cases / Limitations

RLHF can encode labeler bias, encourage verbose answers, produce reward hacking, reduce diversity, and fail on rare or technical prompts. It improves preference alignment, not guaranteed truth.

14. Variations

RLAIF uses AI feedback instead of human feedback. Constitutional AI uses rule-based critique and revision. DPO removes explicit RL. IPO/KTO/ORPO are direct preference optimization variants. PPO-style RLHF remains important for research and large post-training stacks.

15. Related Topics

SFT teaches imitation; RLHF optimizes preference. DPO is a simpler alternative. Reward modeling connects to ranking. Safety training connects to refusal behavior and red teaming.

16. Interview Questions

1. What problem does RLHF solve? It aligns outputs with human preferences when labels are subjective.
2. Why train a reward model? Human scoring every RL sample is too expensive.
3. What is the KL penalty for? It prevents the policy from drifting too far from a stable reference.
4. Why use pairwise preferences? Ranking is easier and more reliable than absolute scoring.
5. What is reward hacking? The model exploits the learned reward instead of genuinely improving.
6. Is RLHF enough for factuality? No, it improves preferred behavior but does not guarantee truth.
7. What comes before RLHF? Pretraining and usually supervised fine-tuning.
8. What is PPO's role? It updates the policy with controlled steps.
9. How do you evaluate RLHF? Human win rate, safety tests, benchmark regressions, and reward-model diagnostics.
10. What is a reference model? A frozen baseline used for KL regularization.

17. Practice Tasks

Implement pairwise reward loss. Build a small preference dataset for summaries. Compare SFT output vs reward-ranked output. Debug a reward model that always prefers longer answers. Extend with DPO.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Summary Preference Trainer	Trains reward model for summaries	PyTorch, Transformers	TL;DR preference data	Shows alignment basics
Helpful Chat Ranker	Ranks chatbot answers	HF, PEFT	Anthropic HH subset	Practical eval pipeline
Reward Hacking Demo	Shows over-optimization failure	PyTorch	Synthetic prompts	Research maturity

19. Quick Revision

Key idea: optimize LLM behavior using human preferences. Main formula: reward minus KL penalty. Use when response quality is subjective. Metrics: win rate, reward accuracy, safety pass rate. Trap: trusting reward score alone. Interview one-liner: RLHF turns human preferences into a reward signal for post-training.

20. Final Cheat Sheet

Definition: preference-based alignment. Input/output: prompt to preferred response. Main steps: SFT, preference data, reward model, PPO. Hyperparameters: KL beta, learning rate, rollout length. Pros: better helpfulness. Cons: costly, reward hacking. Best use: chat alignment.

DPO

1. Overview

Direct Preference Optimization (DPO) trains an LLM directly from chosen/rejected preference pairs without training a separate reward model or running online RL. It is popular because it is simpler, more stable, and cheaper than PPO-based RLHF for many alignment tasks.

2. Intuition

If humans prefer answer A over B for the same prompt, increase the model's probability of A and decrease the probability of B, while staying close to a reference model. DPO bakes the reward-model math into a supervised-looking loss.

3. Prerequisites

Cross-entropy training, log probabilities, preference pairs, KL regularization, SFT, token-level likelihoods.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Chosen/rejected	Preferred and non-preferred response	Main training signal	Good answer vs bad answer	Pairwise learning
Reference model	Usually frozen SFT model	Controls drift	π_{ref}	Why not train only chosen?
Log-prob ratio	Preference strength under model vs reference	Core DPO signal	$\log \pi(y_w) - \log \pi(y_l)$	Derive loss intuition
Beta	Controls deviation from reference	Higher beta sharpens preference pressure	$\beta = 0.1$	Tuning stability

5. Algorithm / Working Process

1. Start with an SFT model and freeze a copy as reference.
2. For each prompt, store chosen and rejected answers.
3. Compute policy log probability of both answers.
4. Compute reference log probability of both answers.
5. Optimize DPO loss to prefer chosen over rejected relative to reference.

6. Evaluate with preference win rate and task benchmarks.

6. Mathematical Foundation

DPO loss:

$$L_{DPO} = -E[\log \sigma(\beta((\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) - (\log \pi_{ref}(y_w|x) - \log \pi_{ref}(y_l|x))))]$$

It increases the relative probability of the preferred answer more than the reference model does.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

def dpo_loss(policy_chosen, policy_rejected, ref_chosen, ref_rejected, beta=0.1):
    policy_logratios = policy_chosen - policy_rejected
    ref_logratios = ref_chosen - ref_rejected
    logits = beta * (policy_logratios - ref_logratios)
    return -F.logsigmoid(logits).mean()

loss = dpo_loss(
    torch.tensor([-12.0, -8.0]),
    torch.tensor([-15.0, -7.5]),
    torch.tensor([-13.0, -8.2]),
    torch.tensor([-14.0, -7.7]),
)
print(loss.item())
```

8. Code Explanation

The model compares chosen and rejected sequence log probabilities. The reference comparison is subtracted, so the policy is rewarded only for improving preference relative to the baseline.

9. Training / Evaluation

Use clean preference pairs. Split by prompt. Track DPO loss, chosen-vs-rejected accuracy, validation win rate, length bias, and benchmark regressions. Tune beta and learning rate carefully.

10. Complexity and Cost

DPO is cheaper than PPO RLHF because it is offline and avoids rollout generation during training. It still needs two forward passes through policy and reference, or cached reference logprobs.

11. Common Use Cases

Chat alignment, style tuning, summarization preference tuning, code assistant response ranking, domain assistant refinement.

12. Common Mistakes

Bad preference data, forgetting the reference model, using token logprobs incorrectly, comparing prompts with different responses without masking, overfitting small preference sets, ignoring length bias.

13. Edge Cases / Limitations

DPO depends heavily on preference quality. It may be weaker than RL for tasks needing exploration or external rewards. It can still reduce diversity or overfit style preferences.

14. Variations

IPO changes the preference objective. KTO uses desirable/undesirable examples without strict pairs. ORPO combines supervised and preference losses. SimPO removes the explicit reference in some formulations.

15. Related Topics

DPO vs RLHF: DPO is offline and simpler; RLHF uses reward model plus RL. DPO vs SFT: SFT imitates chosen answers; DPO learns relative preferences.

16. Interview Questions

1. What is DPO? Direct training on preference pairs.
2. Why is it simpler than RLHF? No reward model or PPO loop.
3. What data does DPO need? Prompt, chosen response, rejected response.
4. Why use a reference model? To regularize behavior and preserve capabilities.
5. What does beta do? Controls preference strength vs reference closeness.
6. How is DPO evaluated? Win rate and downstream benchmarks.
7. Does DPO need online sampling? No, it is usually offline.
8. Can DPO replace SFT? Usually no; it commonly follows SFT.
9. What is the risk of noisy pairs? The model learns wrong preferences.
10. Why compute sequence logprob? Preference applies to full response likelihood.

17. Practice Tasks

Implement DPO loss. Cache reference logprobs. Fine-tune a small model on synthetic preference pairs. Analyze length bias. Compare SFT-only vs SFT+DPO.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
DPO Chat Tuner	Fine-tunes small chat model	TRL, PEFT	HH-RLHF subset	Modern alignment skill
Style Preference Model	Tunes concise answers	HF	Custom pairs	Product relevance
DPO Loss Lab	Visualizes beta effects	PyTorch, Streamlit	Synthetic	Math clarity

19. Quick Revision

Key idea: increase probability of preferred response relative to rejected and reference. Formula: DPO sigmoid loss on log-ratio difference. Use when you have preference pairs. Trap: bad logprob masking.

20. Final Cheat Sheet

Definition: offline preference optimization. Input/output: preference pairs to aligned model. Steps: compute policy/ref logprobs, optimize DPO loss. Hyperparameters: beta, LR, batch size. Pros: simpler than RLHF. Cons: data-sensitive.

LoRA

1. Overview

Low-Rank Adaptation (LoRA) fine-tunes large neural networks by freezing original weights and training small low-rank update matrices. It is widely used to adapt LLMs cheaply for domains, styles, tasks, and instruction tuning.

2. Intuition

Instead of changing a huge matrix directly, learn a small correction that can be written as the product of two skinny matrices. It is like adding a small steering attachment to a large machine instead of rebuilding the machine.

3. Prerequisites

Matrix multiplication, rank, linear layers, Transformer attention, fine-tuning, PyTorch modules.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Frozen base	Original model weights are not trained	Saves memory	Llama weights frozen	Why memory drops
Low-rank update	$\Delta W = BA$	Few trainable params	rank 8 adapter	Rank tradeoff
Target modules	Layers receiving LoRA	Controls quality/cost	q_proj, v_proj	Where to apply LoRA
Merge	Add LoRA delta into base weights	Faster inference	merged checkpoint	Deployment

5. Algorithm / Working Process

For a linear layer $y = Wx$, freeze W . Add trainable matrices $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$. During forward pass, compute $y = Wx + \alpha/r \cdot BAx$. Train only A and B .

6. Mathematical Foundation

$$W' = W + \Delta W, \quad \Delta W = \frac{\alpha}{r} B A$$

If W has shape $k \times d$, LoRA trains $r(d + k)$ parameters instead of kd . With $r \ll \min(d, k)$, parameter savings are large.

7. Practical Implementation

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("gpt2")

config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["c_attn"],
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
)

model = get_peft_model(model, config)
model.print_trainable_parameters()
```

8. Code Explanation

`r` is adapter rank. `lora_alpha` scales the update. `target_modules` chooses which linear layers receive adapters. `get_peft_model` freezes base weights and inserts trainable LoRA modules.

9. Training / Evaluation

Prepare instruction or domain data. Use normal language-model loss. Evaluate task accuracy, perplexity, human preference, latency, and whether the adapter overfits. Tune rank, alpha, dropout, target modules, and learning rate.

10. Complexity and Cost

Training memory is much lower than full fine-tuning because gradients and optimizer states are stored only for adapters. Inference has slight overhead unless adapters are merged.

11. Common Use Cases

Domain adaptation, instruction tuning, style tuning, personalization, multilingual adaptation, code assistant tuning.

12. Common Mistakes

Targeting wrong module names, using too high rank for small data, forgetting to save adapter config, evaluating only training loss, mixing incompatible base model and adapter, expecting LoRA to add missing knowledge perfectly.

13. Edge Cases / Limitations

LoRA may underperform full fine-tuning for large distribution shifts. It still needs quality data. Many adapters loaded at once can complicate serving.

14. Variations

AdaLoRA changes rank adaptively. DoRA separates magnitude and direction. QLoRA combines LoRA with quantized base weights. IA3 trains multiplicative vectors instead of low-rank matrices.

15. Related Topics

LoRA vs full fine-tuning: cheaper but less flexible. LoRA vs QLoRA: QLoRA quantizes base model too. LoRA is a PEFT method.

16. Interview Questions

1. What does LoRA train? Low-rank adapter matrices.
2. Why freeze base weights? To reduce memory and preserve base capability.
3. What is rank r ? Size of the adapter bottleneck.
4. What is alpha? Scaling factor for LoRA update.
5. Can LoRA be merged? Yes, add delta to base weights.
6. Where is LoRA applied in LLMs? Attention and sometimes MLP projections.
7. Does LoRA reduce inference memory? Adapter is small; base still needed.
8. Why is it parameter efficient? It trains $r(d + k)$ not dk parameters.
9. What happens if r is too small? Underfitting.
10. What happens if r is too large? More memory and overfitting risk.

17. Practice Tasks

Fine-tune GPT-2 with LoRA. Compare ranks 4, 8, 16. Merge adapter and test generation. Inspect trainable parameter count. Debug wrong `target_modules`.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain LoRA Bot	Tunes model for one domain	PEFT, Transformers	Company FAQ	Practical fine-tuning
Rank Ablation Study	Compares LoRA ranks	PyTorch	Alpaca subset	Research mindset
Multi-Adapter Demo	Switches between styles	PEFT	Custom style data	Deployment skill

19. Quick Revision

Key idea: train low-rank deltas while base is frozen. Formula: $W' = W + \alpha BA/r$. Use for cheap fine-tuning. Trap: wrong target modules.

20. Final Cheat Sheet

Definition: parameter-efficient fine-tuning via low-rank updates. Input/output: base model plus task data to adapter. Hyperparameters: r , α , dropout, target layers. Pros: cheap. Cons: limited capacity.

QLoRA

1. Overview

Quantized LoRA (QLoRA) fine-tunes a quantized base model using LoRA adapters. The base model is stored in low precision, commonly 4-bit, while adapter weights train in higher precision. This enables fine-tuning large LLMs on limited GPU memory.

2. Intuition

Keep the heavy model compressed and frozen, then train a small adapter on top. It is like reading a compressed reference book and writing small sticky-note corrections.

3. Prerequisites

LoRA, quantization, mixed precision, GPU memory, optimizer states, Transformers, PEFT.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
4-bit base	Frozen model stored compactly	Huge memory saving	NF4 weights	Why QLoRA fits on one GPU
LoRA adapters	Trainable small matrices	Adapt task behavior	rank 16	Why not train quantized weights
Double quantization	Quantizes quantization constants	Saves extra memory	bitsandbytes	Memory details
Paged optimizer	Handles memory spikes	Prevents OOM	paged AdamW	Practical training

5. Algorithm / Working Process

Load base model in 4-bit. Insert LoRA adapters. During forward pass, dequantize needed weights for computation, apply LoRA update, compute loss, and update only adapter parameters. Save adapter, not full model.

6. Mathematical Foundation

Quantized base:

$$W_q = Q(W), \quad W \approx \text{dequantize}(W_q)$$

LoRA update:

$$y = \text{dequantize}(W_q)x + \frac{\alpha}{r}BAx$$

NF4 is designed for normally distributed neural weights and gives better 4-bit representation than uniform quantization.

7. Practical Implementation

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)

model = AutoModelForCausalLM.from_pretrained(
    "gpt2",
    quantization_config=bnb_config,
    device_map="auto",
)

lora = LoraConfig(r=8, lora_alpha=16, target_modules=["c_attn"],
    task_type="CAUSAL_LM")
model = get_peft_model(model, lora)
model.print_trainable_parameters()
```

8. Code Explanation

`load_in_4bit` compresses the frozen base. `nf4` is the 4-bit quantization type. LoRA adds trainable adapters. Only adapter parameters receive gradients.

9. Training / Evaluation

Use instruction data, pack sequences carefully, and monitor memory. Evaluate generation quality, task metrics, adapter overfitting, and latency. Compare with LoRA on higher precision if resources allow.

10. Complexity and Cost

QLoRA greatly reduces memory. Compute may be slower than pure fp16 because of quantization overhead. It enables large-model fine-tuning on consumer GPUs.

11. Common Use Cases

Fine-tuning 7B/13B/70B models with limited GPUs, domain-specific assistants, academic experiments, instruction tuning.

12. Common Mistakes

Trying to update quantized base weights, using unsupported GPU kernels, bad compute dtype, forgetting gradient checkpointing for long context, expecting 4-bit training to equal full fine-tuning always.

13. Edge Cases / Limitations

Quantization can hurt fragile tasks. Some layers may need higher precision. Hardware/kernel support matters. Very small datasets still overfit.

14. Variations

8-bit LoRA is less compressed but often stable. LoftQ initializes LoRA for quantized models. AWQ/GPTQ are more inference-oriented quantization methods.

15. Related Topics

QLoRA = quantization + LoRA + PEFT. It differs from post-training quantization because adapters are trained.

16. Interview Questions

1. What is QLoRA? LoRA fine-tuning on a quantized frozen base.
2. Why use 4-bit? To reduce memory.
3. Are base weights trained? No.
4. What is NF4? A 4-bit format suited to neural weights.
5. Why use bf16 compute? Stability during operations.
6. How does QLoRA differ from LoRA? Base model is quantized.
7. What is double quantization? Quantizing scale constants too.
8. Main risk? Quality loss or kernel incompatibility.
9. What is saved? Adapter weights and config.
10. Best use case? Large-model adaptation on limited hardware.

17. Practice Tasks

Load a small model in 4-bit. Add LoRA. Measure memory. Fine-tune on tiny instruction data. Compare generated outputs with base model.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Single-GPU Fine-Tuner	Tunes a chat model cheaply	PEFT, bitsandbytes	Alpaca subset	Practical LLM training
Quantization Study	Compares 4-bit vs 8-bit	HF	WikiText	Systems awareness
Domain Adapter	Legal/medical-style assistant	QLoRA	Public QA data	Applied AI project

19. Quick Revision

Key idea: quantize base, train adapters. Formula: dequantized base output plus LoRA delta. Use when GPU memory is limited. Trap: unsupported quantization stack.

20. Final Cheat Sheet

Definition: memory-efficient LoRA with quantized base. Input/output: quantized model plus data to adapter. Hyperparameters: rank, alpha, quant type, compute dtype. Pros: cheap. Cons: quantization overhead.

PEFT

1. Overview

Parameter-Efficient Fine-Tuning (PEFT) adapts large models by training only a small number of parameters. It is useful when full fine-tuning is too expensive or when many task-specific adapters must share one base model.

2. Intuition

Instead of rewriting the whole model, attach small trainable parts or tune a small subset of parameters. The base model remains a reusable foundation.

3. Prerequisites

Fine-tuning, Transformers, gradients, adapters, prompts, low-rank matrices.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Adapter	Small trainable module	Cheap task adaptation	LoRA adapter	Serving many tasks
Frozen backbone	Shared base model	Saves storage and compute	Llama base	Avoid catastrophic forgetting
Prompt tuning	Train soft prompt embeddings	Very small parameter count	20 virtual tokens	When it works
Prefix tuning	Train key/value prefixes	Influences attention	Prefix vectors	Difference from prompt tuning

5. Algorithm / Working Process

Choose PEFT method, freeze base model, add trainable parameters, train on task data, save small adapter, load adapter with base model at inference.

6. Mathematical Foundation

General PEFT:

$$f_{\theta,\phi}(x), \quad \theta \text{ frozen}, \quad \phi \text{ trainable}, \quad |\phi| \ll |\theta|$$

Optimize:

$$\min_{\phi} L(f_{\theta,\phi}(x), y)$$

7. Practical Implementation

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForSequenceClassification

base = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased")
config = LoraConfig(r=4, lora_alpha=8, target_modules=["q_lin", "v_lin"],
task_type="SEQ_CLS")
model = get_peft_model(base, config)
model.print_trainable_parameters()
```

8. Code Explanation

The base classifier is loaded normally. PEFT inserts LoRA modules into attention projections. Training updates only the new parameters.

9. Training / Evaluation

Use the same task metrics as normal fine-tuning: accuracy/F1 for classification, ROUGE/BLEU for generation, exact match for QA, win rate for chat. Validate adapter quality and check for base-model compatibility.

10. Complexity and Cost

PEFT reduces optimizer-state memory and checkpoint size. Full base model is still needed for inference. Adapter switching is cheap compared with loading full model copies.

11. Common Use Cases

Multi-tenant LLM serving, task-specific models, domain adaptation, personalization, low-resource research, edge experiments.

12. Common Mistakes

Assuming PEFT always matches full fine-tuning, training too few parameters for a large shift, poor adapter management, incompatible model architecture names, bad validation splits.

13. Edge Cases / Limitations

Large domain shifts or new skills may require full fine-tuning or continued pretraining. Soft prompts can be brittle. Adapter composition can conflict.

14. Variations

LoRA, QLoRA, adapters, prefix tuning, prompt tuning, IA3, BitFit. LoRA/QLoRA are most placement-relevant.

15. Related Topics

PEFT vs full fine-tuning: cheaper but less expressive. PEFT vs RAG: PEFT changes behavior; RAG injects knowledge at inference.

16. Interview Questions

1. What is PEFT? Fine-tuning a small parameter subset.
2. Why use it? Memory and storage savings.
3. Name PEFT methods. LoRA, adapters, prompt tuning, prefix tuning.
4. Does PEFT need base model at inference? Yes.
5. Is LoRA PEFT? Yes.
6. What is adapter merging? Folding adapter deltas into base weights.
7. PEFT or RAG for new facts? Usually RAG.
8. PEFT or full fine-tuning for new style? PEFT often works.
9. Main limitation? Limited adaptation capacity.
10. How evaluate? Same metrics as target task plus regression tests.

17. Practice Tasks

Train LoRA for classification. Compare full fine-tuning vs PEFT memory. Save/load an adapter. Try prompt tuning on a small model. Measure checkpoint size.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Adapter Zoo	Multiple adapters on one base	PEFT	GLUE tasks	Efficient deployment
PEFT Benchmark	Compares methods	HF	SST-2, AG News	Interview depth
Personal Writing Adapter	Tunes style	LoRA	Own writing samples	Portfolio demo

19. Quick Revision

Key idea: freeze base, train small additions. Formula: optimize ϕ while θ frozen. Use when resources are limited. Trap: expecting it to learn large new knowledge reliably.

20. Final Cheat Sheet

Definition: small-parameter adaptation. Input/output: base model plus task data to adapter. Hyperparameters: method, rank/prefix length, LR. Pros: cheap. Cons: lower capacity.